Lecture 15: Intro to Machine Learning

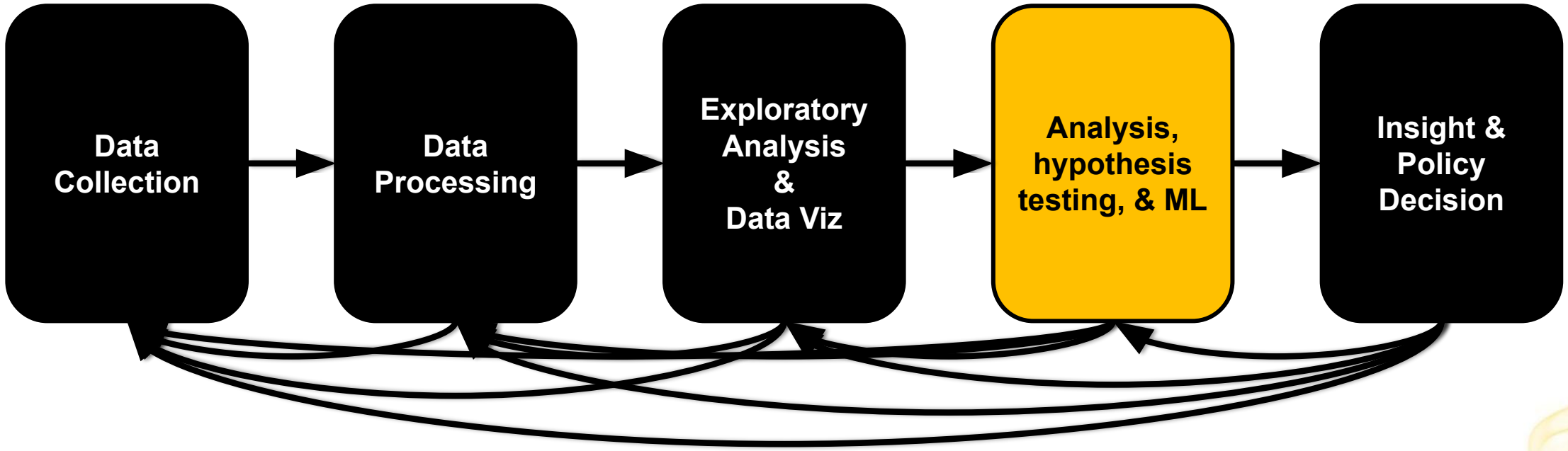
CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA4, WA4 due this Sunday, March 29
- Office hours remain the same:
  - Anna on Zoom: Tue 11 am–12 pm
  - TAs in person (IRB 2216): Tue 12-1 pm, Fri 9-10 am, Fri 4-5 pm
  - TAs on Zoom: Thu 11 am–12pm

Intro to Machine Learning


Data Collection
Data Processing
Exploratory Analysis & Data Viz
Analysis, hypothesis testing, & ML
Insight & Policy Decision

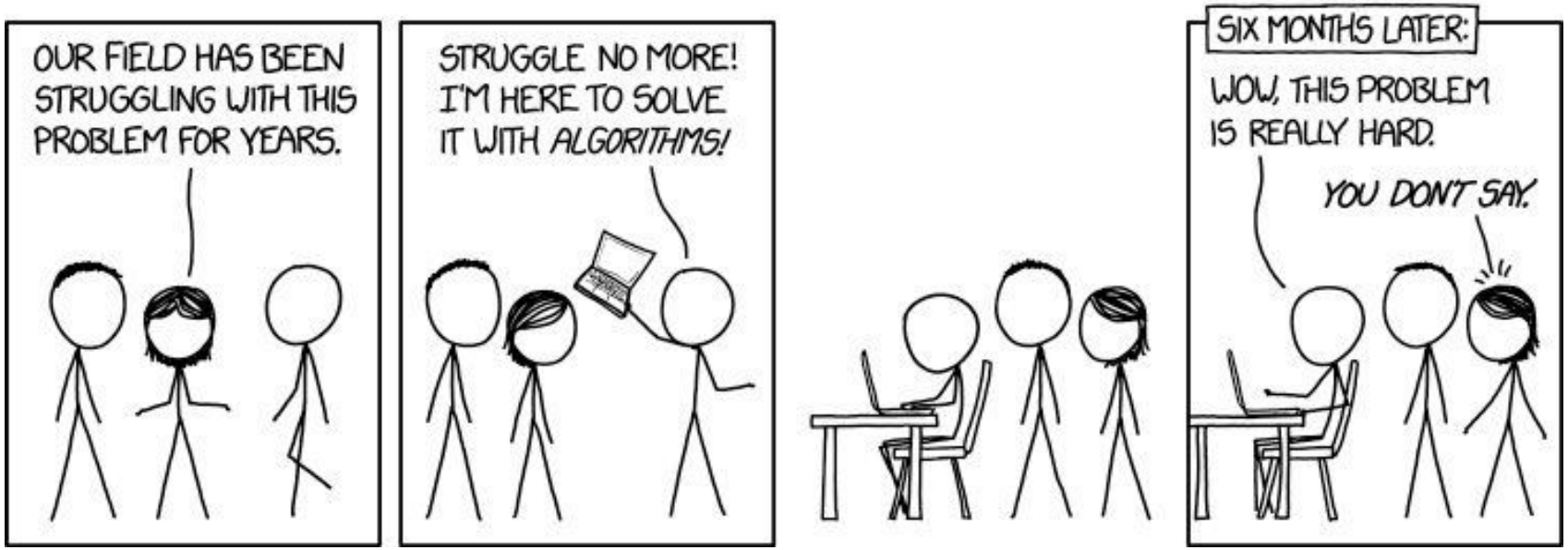
“**Learning** is any process by which a system **improves performance from experience**.”

- *Herbert Simon - political scientist, computer scientist, economist*

Humans Learn

Machines can learn, too!

Or at least, algorithms running on machines can



What is Machine Learning (ML)

A subfield of artificial intelligence (AI)

- Focuses on development and application of **algorithms** and **statistical models**
- To enable computer systems to **learn and improve** their performance on a specific task or problem

In other words, ML enables computers to make predictions or decisions based on patterns and insights derived from data.

This is particularly useful for pattern recognition and predictive analysis.

Example: Is e-mail spam?

Task (classification): Assign a **label** “Spam” or “Not Spam” to email observations.
Training data (supervised): A dataset consisting of example e-mails and their attributes along with pre-assigned labels for each observation.
Algorithm: Naive Bayes
Model: A classifier that predicts the “Spam” or “Not Spam” label for any new input (a previously unseen e-mail observation)

Big picture (for supervised learning → classification)

We have a sample from a population. We know some information about the data points (e.g. age, high school GPA, SAT scores). We also know a label: whether this person got into UMD.

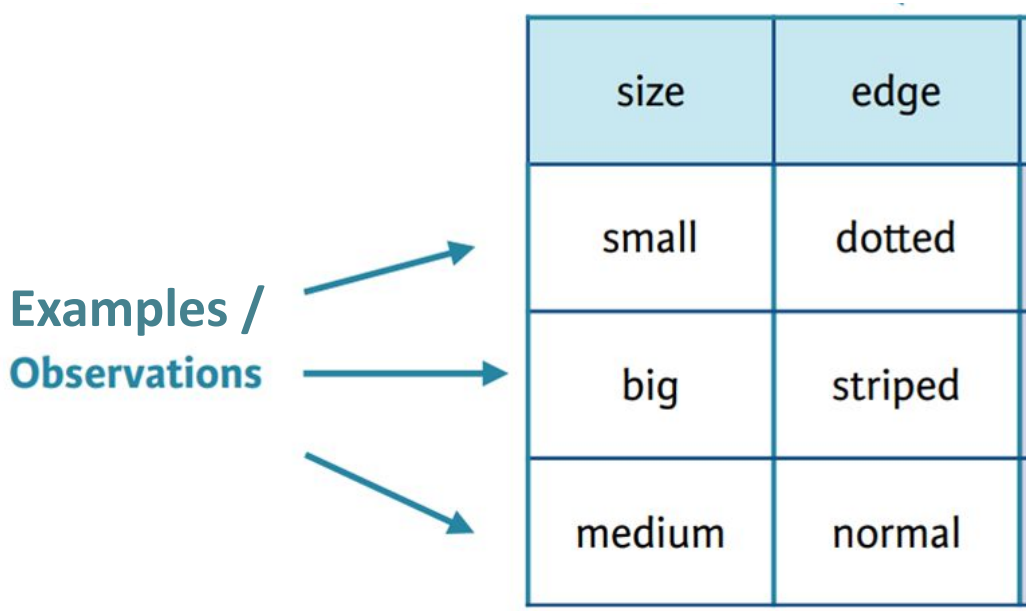
We can look at this data closely and try to see what influences whether the label is true or false.

Now, we have a new data point *from the same population*. What label can we predict for this data point given the patterns we learned from the sample?

Basic Terminology

Examples / Observations

Examples, or observations, are the records (ex. rows) in the input dataset.

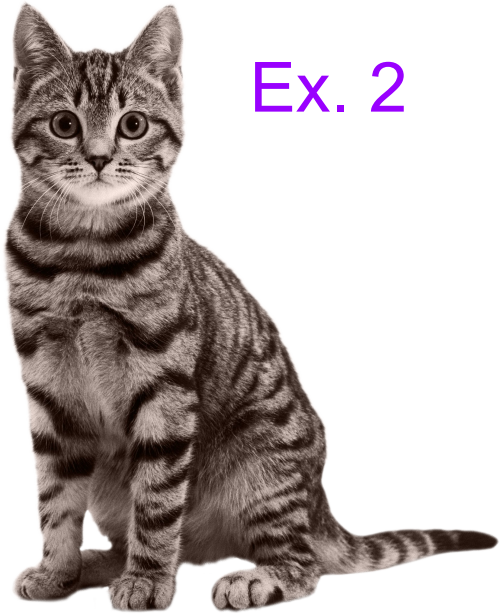


Examples / Observations (cont.)

The individual things you want to learn from (and later make predictions about similar things).

Ex. 1

Ex. 2



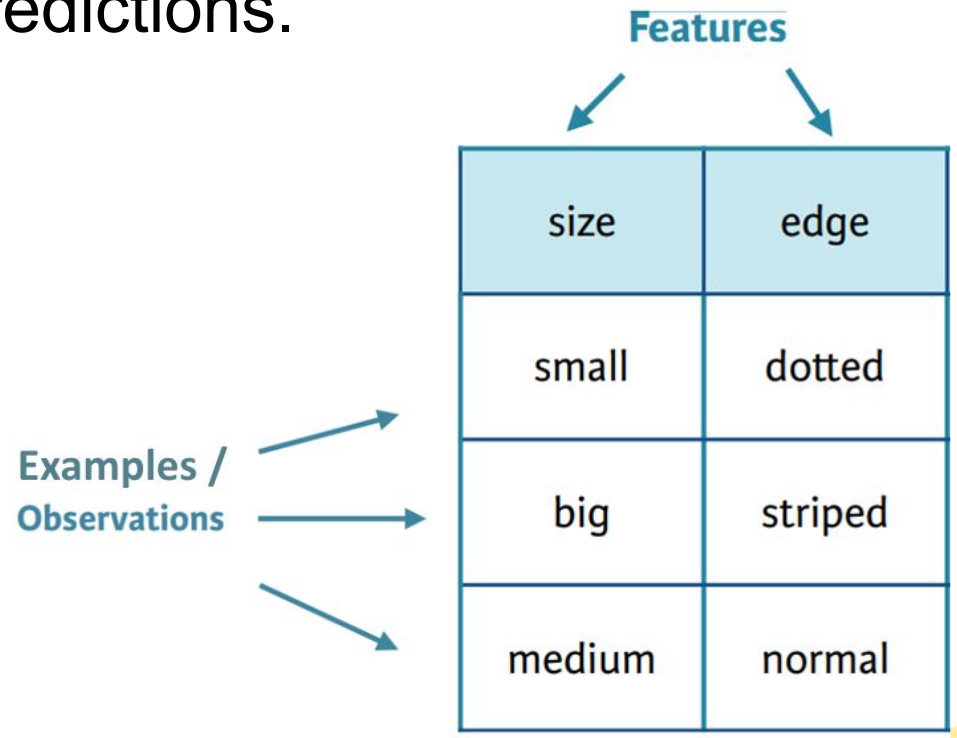
Ex. 3

Ex. 4

Features / Attributes / Dimensions

Features (a.k.a. attributes, dimensions, variables, or columns), are the **variable inputs** that machine learning (ML) models use during training and inference to make predictions.

These are the predictor variables that determine the model's output.



Features / Attributes / Dimensions (cont.)

In plain language: **the properties of the thing you want to learn about**.

Age (yrs): 1
Color: Orange and white
Pattern: Striped
Ears: Pointy
Weight (lbs): 5
Eye color: Green
Has Tail: true

Dimensionality

The **number of dimensions** (a.k.a. features, attributes, variables, etc) in the input dataset.

We'll talk about dimensionality a lot more later on.

Vectors

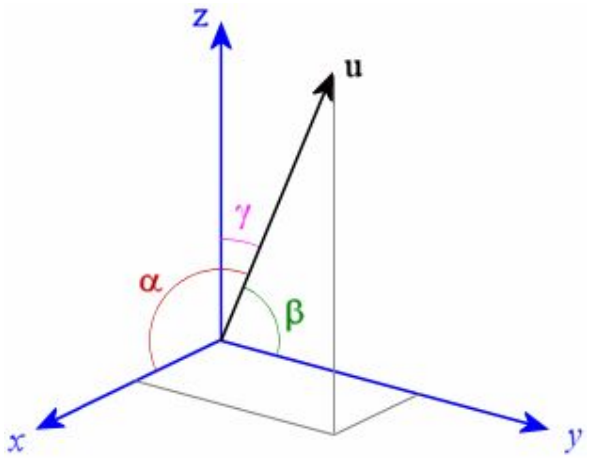
In classical mathematics and physics, a vector (visualized as an arrow), provides 2 things: a **magnitude** and a **direction**.

In ML, vectors are mathematical representations of data that also have both **magnitude** and **direction**.

- They are numerical arrays where numbers are placed in order, and each individual number can be identified by its index in that order.
- Typically represented as a column (i.e. a row transposed)

Vectors (cont.)

Example: a vector, ***u***, in 3-dimensional space:



$$\|\vec{u}\| = \sqrt{x^2 + y^2 + z^2}$$

$$\cos(\alpha) = \frac{x}{\|\vec{u}\|}$$

$$\cos(\beta) = \frac{y}{\|\vec{u}\|}$$

$$\cos(\gamma) = \frac{z}{\|\vec{u}\|}$$

```
import numpy as np


u = np.array([1, 2, 3])

print('x:', u[0], 'y:', u[1], 'z:', u[2])
```

```
x: 1 y: 2 z: 3
```

Feature Vector

An *n*-dimensional vector of numerical features that represent an observation (row) in an input dataset.

- Ex. RGB color encoding
  - The feature vector is color = [R, G, B]
  - ex. red = [255, 0, 0]

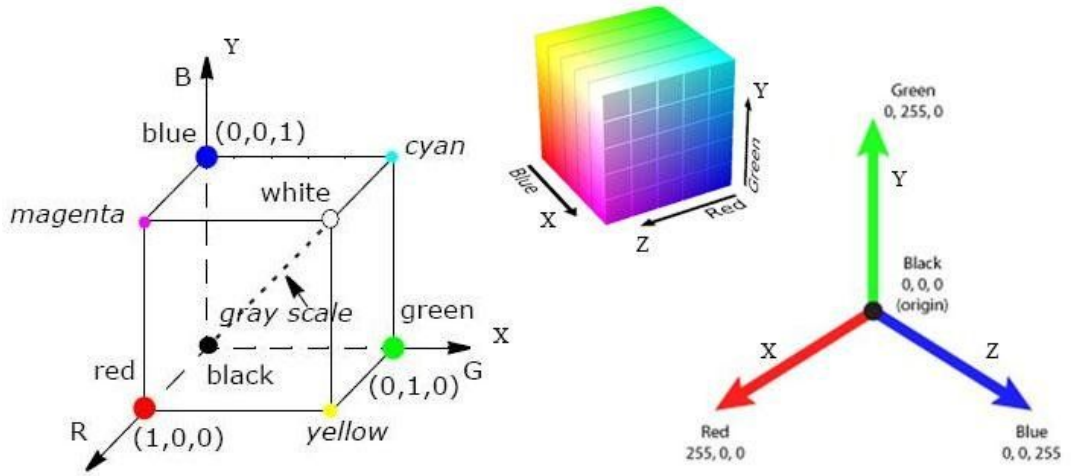
We'll look at more interesting examples later on.

Vector Space

a.k.a. the **Feature Space** (or Feature Set)

The set of all possible vectors within a specific space.

- Ex. RGB vector space
  - The color space of interest is the cube formed by all possible color values



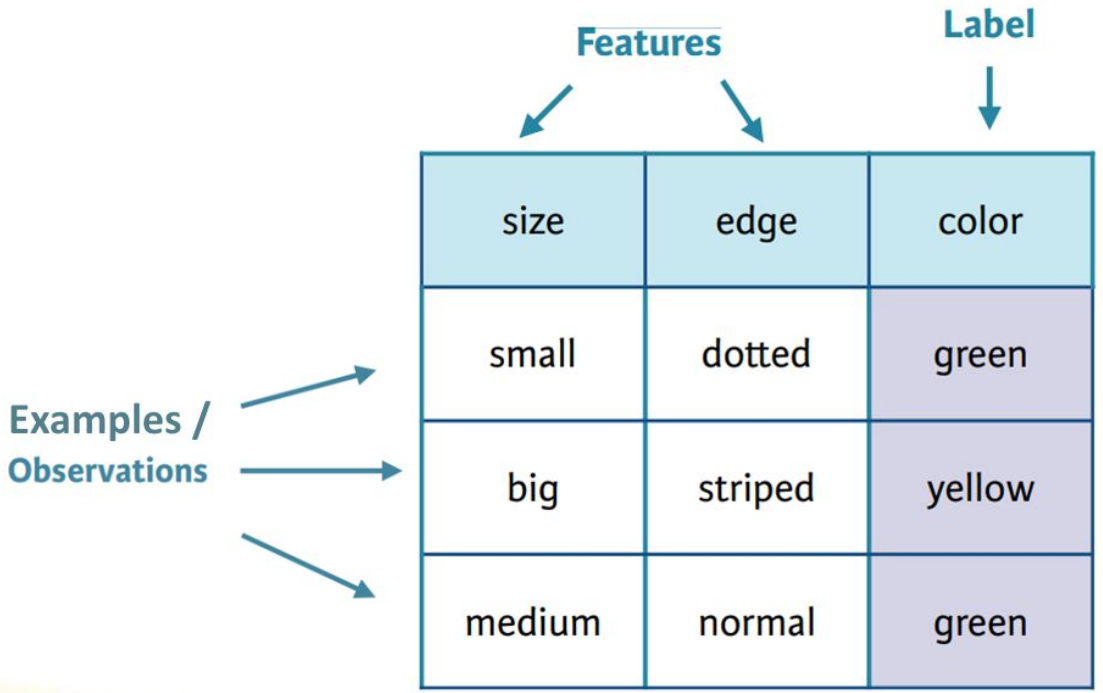
Embeddings

Numerical representations of real-world objects.

In ML, vectors are usually embeddings.

Label

An attribute you want to predict. It is either already assigned (ex. an input dataset that is pre-labeled) or is inferred/predicted by an ML model. It provides context and categorization for raw data.



Label (cont.)

Label → Cat Type: Orange Tabby

Task

A machine learning task is usually a prediction or inference to be made.

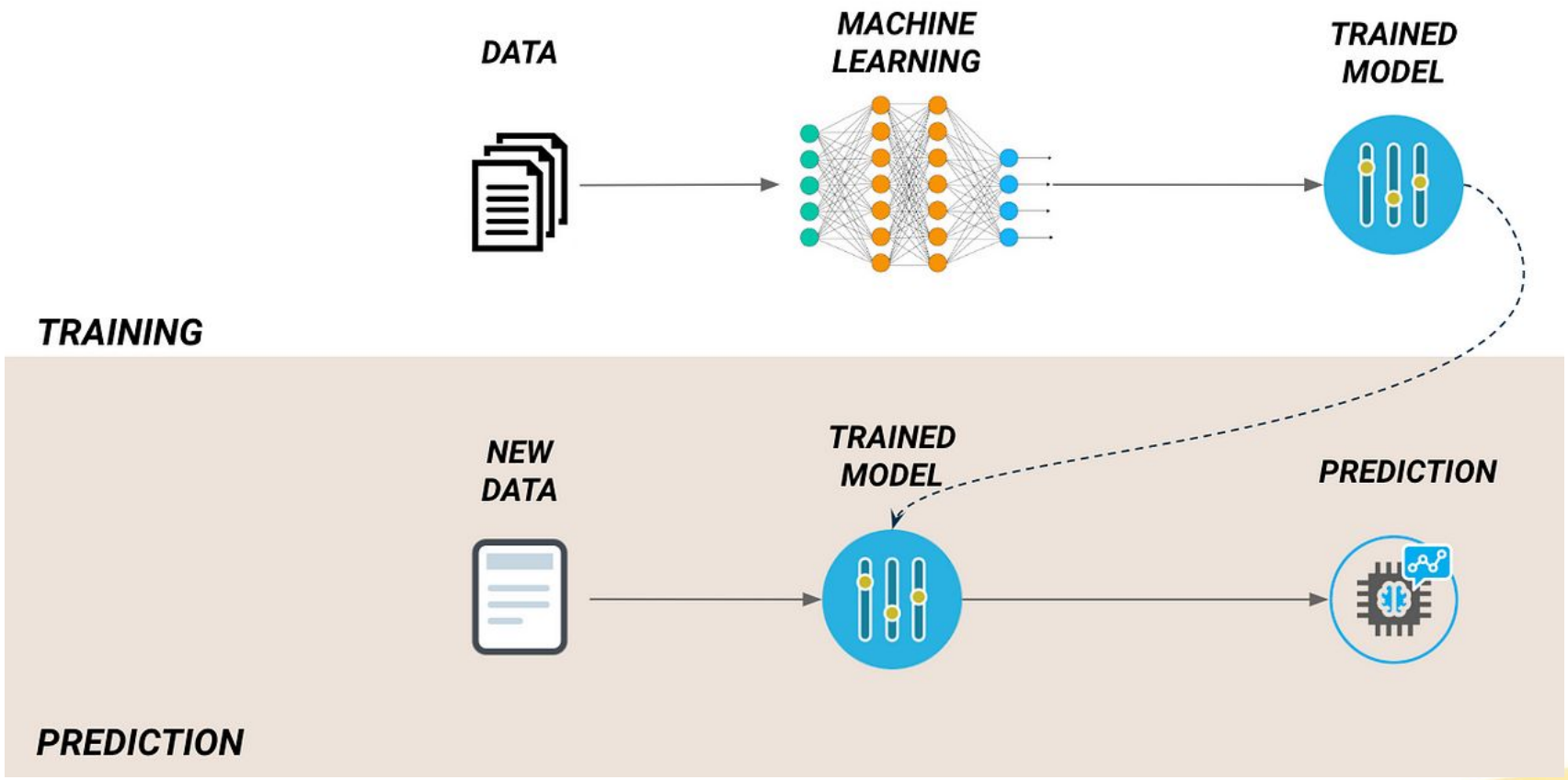
The task depends on the question at hand and the data that is available.

- Ex. Classify this loan applicant as "Will pay loan" or "Will default"

Algorithm vs. Model

- These are often (incorrectly) used interchangeably
- An ML **algorithm *builds* a model** based on sample data (called **training data**).
  - Algorithms are implemented in code and take training data as **input**
    - Ex. Linear regression, KNN, decision trees
  - Models are **output** by the ML algorithm
    - Consist of
      (a) input data
      (b) a prediction algorithm (a procedure for using the input data to make a prediction)
    - Represent what was learned by the ML algorithm
    - A model is an object (usually stored as a file) which can be deployed and used as part of an application or service.

Algorithm vs. Model (cont.)



Example: Is e-mail spam?

Task (classification): Assign a **label** “Spam” or “Not Spam” to email observations.
Training data (supervised): A dataset consisting of example e-mails and their attributes along with pre-assigned labels for each observation.
Algorithm: Naive Bayes
Model: A classifier that predicts the “Spam” or “Not Spam” label for any new input (a previously unseen e-mail observation)

Model Training

The process of **teaching an ML model** to recognize patterns and make predictions. This is done by feeding **training data** to an ML algorithm, which it learns from in order to output the model.

Model Validation

The process of evaluating the performance of a model during training.

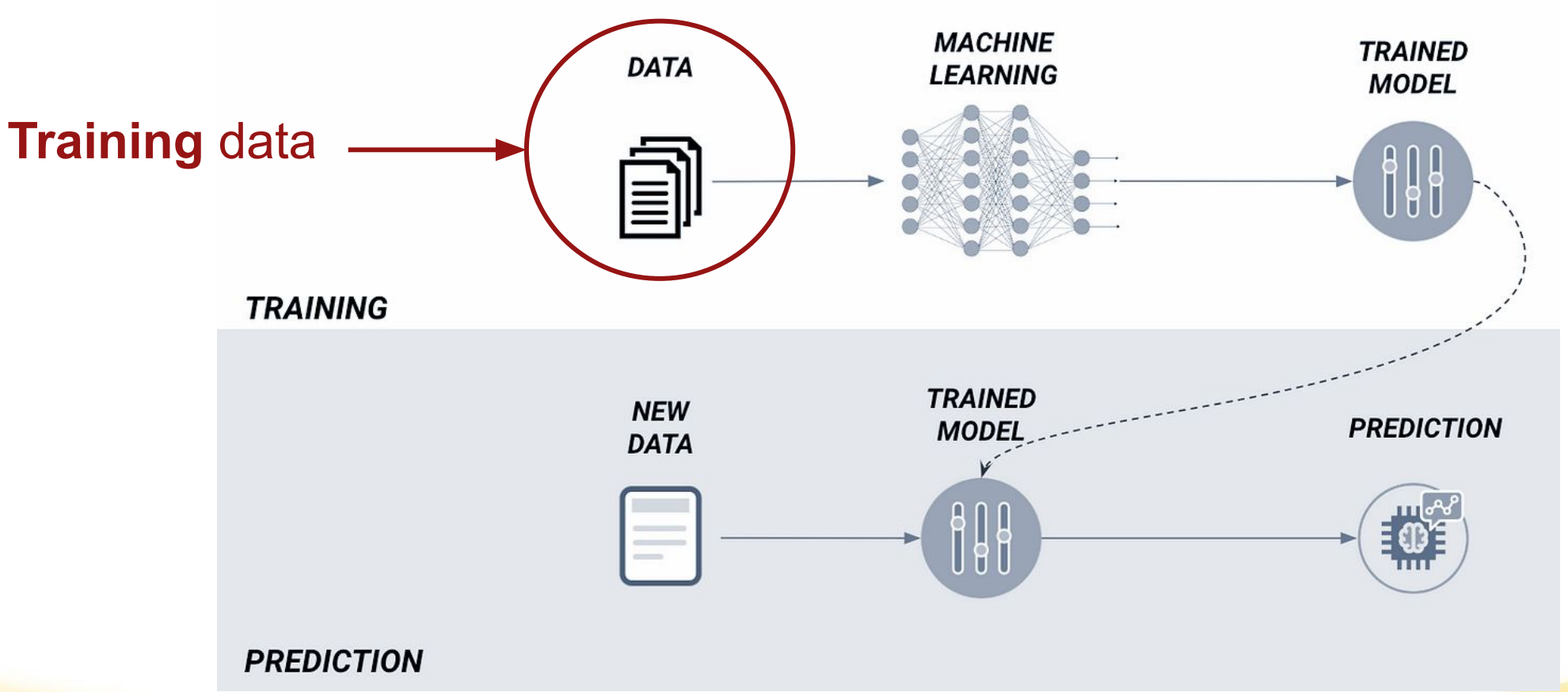
- Evaluates the model
  - Provides an unbiased evaluation of a model fit (*future topic*)
- Aids in
  - Model selection: choosing a model that performs the best on validation data
  - Hyperparameter tuning (*future topic*)

Model Testing

The process of exposing a final ML model to new, previously unseen data to see how it performs.

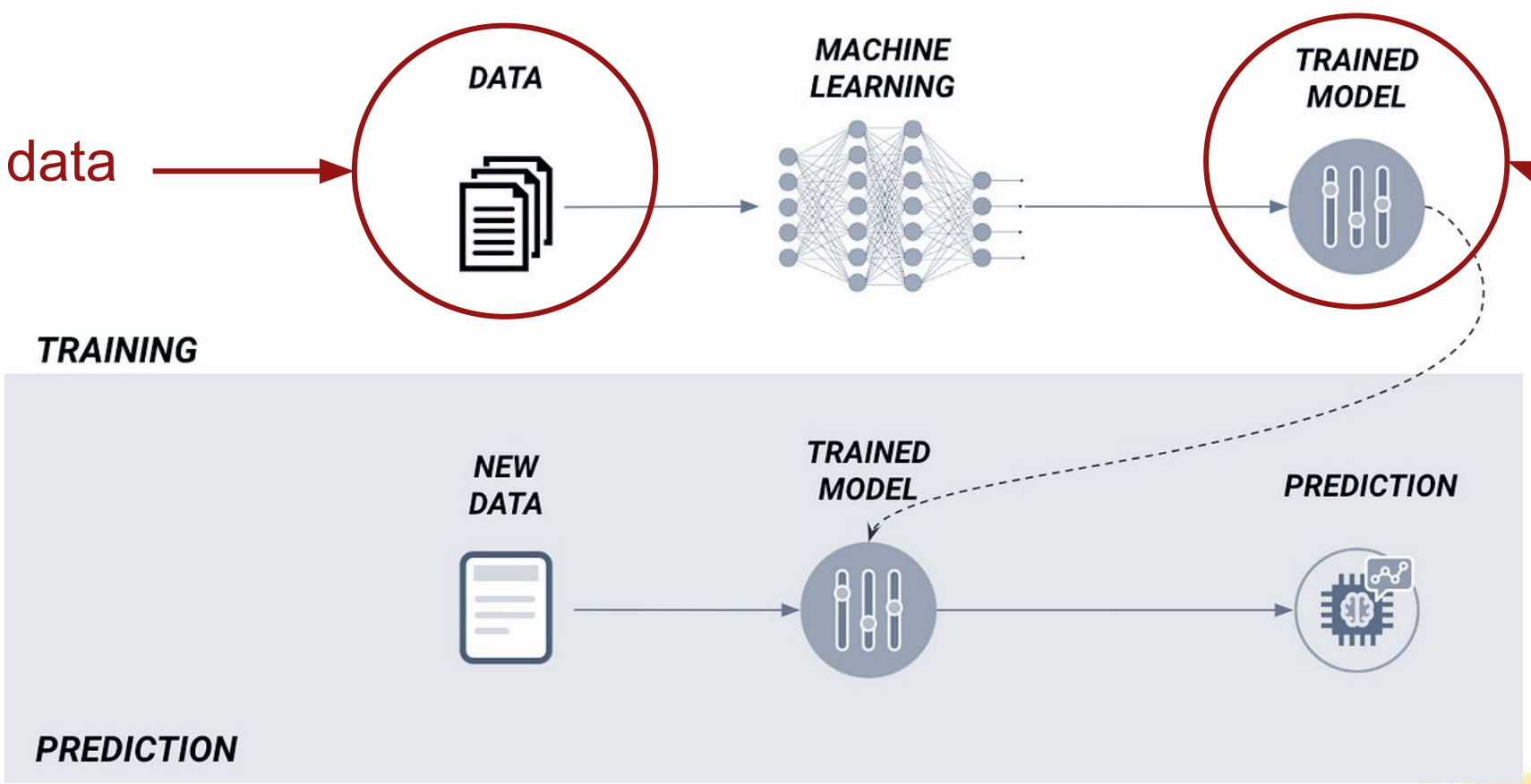
- Evaluates the model's **accuracy**.

Model Training vs. Validation vs. Test



Model Training vs. Validation vs. Test

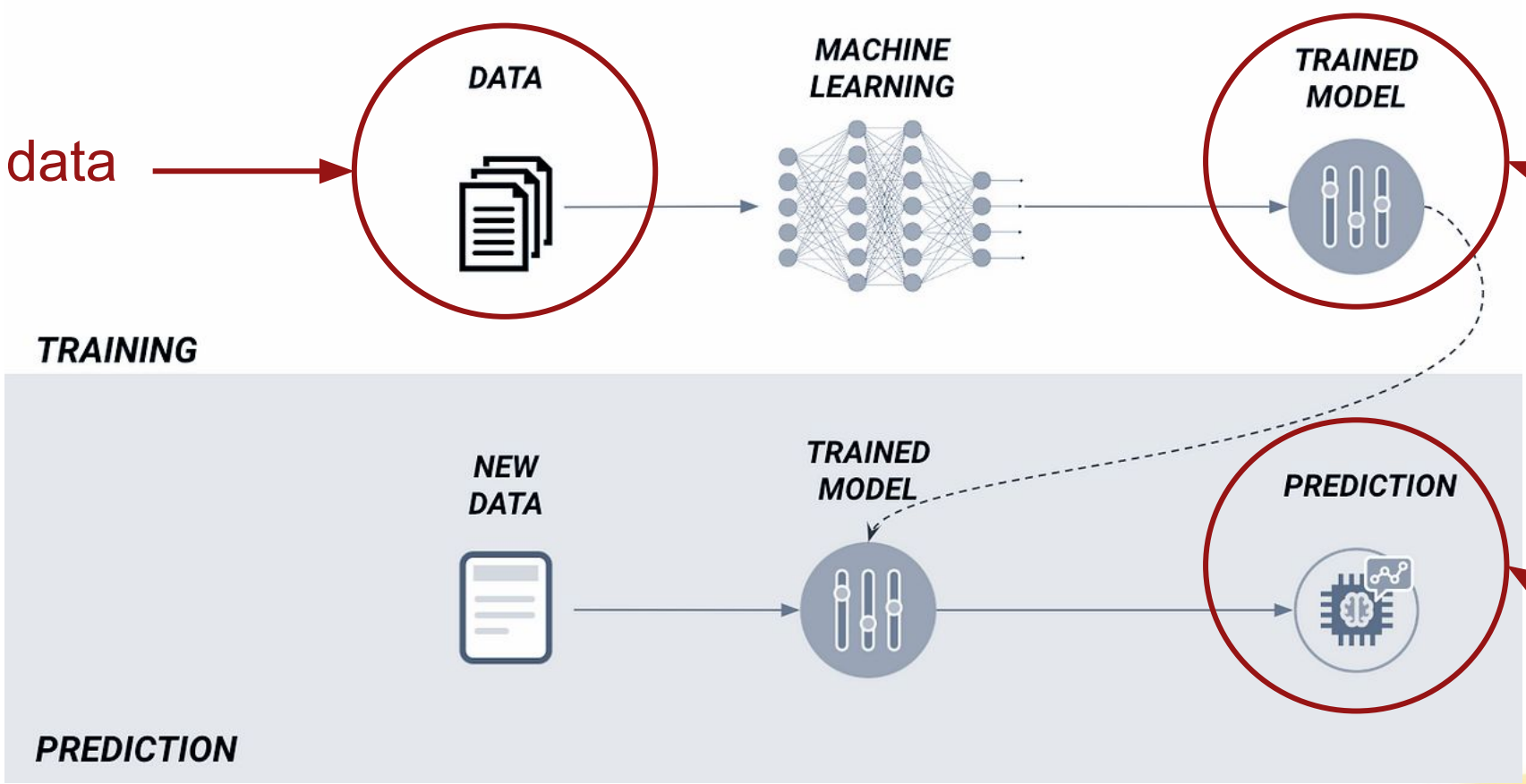
Training data



Validation done here with validation data

(technically part of the training phase, but not used to train the model!)

Model Training vs. Validation vs. Test



Training data

Validation done here with validation data

Model **testing** done here with test data

When developing an algorithm, you have

A dataset. You divide it into:

- A training subset (~75%)
- A validation subset (~10%)
- A testing subset (~15%)

Model Deployment

The process of integrating an ML model into an appropriate environment so it can be used to make predictions (outputs) with new data (inputs).

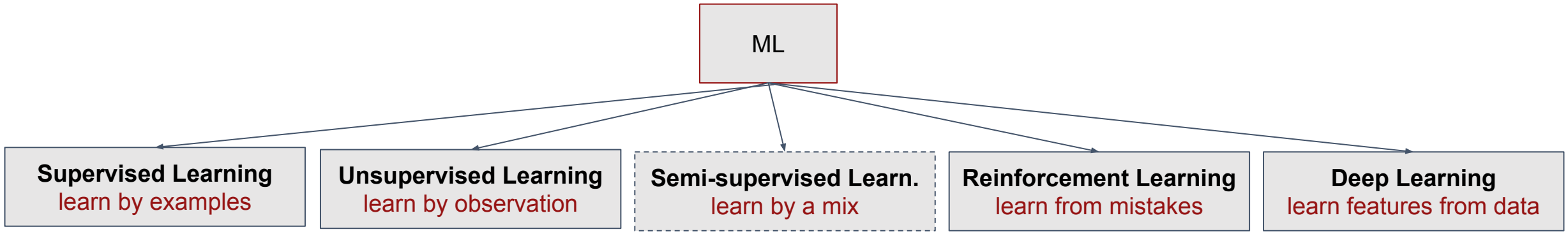
- Depending on the context, **production** (online) models can be deployed to:
  - **Clients**
    - Ex. a mobile device
    - Improved latency, limitation on model size
  - **Services**
    - Ex. Behind an API
    - Added latency, no limit on model size
- They can also be used in **offline** environments
    - Used in non-production environments for **analysis** or **batch processing**

High-Level View

Common classes of ML Problems

Name	What does it do	Example
Regression	Predict continuous numerical values	Predict the future prices of gas
Classification	Determine a label to assign	Is this item a cat, dog, or hamster?
Clustering	Find “like” values	Recommend a similar t-shirt
Dimensionality Reduction	Reduce the number of variables/features in a dataset, preserving key info	Image compression
Anomaly Detection	Identify values that fall outside a learned “normal” (a kind of classification)	Identify a fraudulent credit card charge
Sequence Prediction	Predict the next item in a sequence	Given 2, 4, 6 → predict 8

Main categories of ML Algorithms



Labeled training dataset

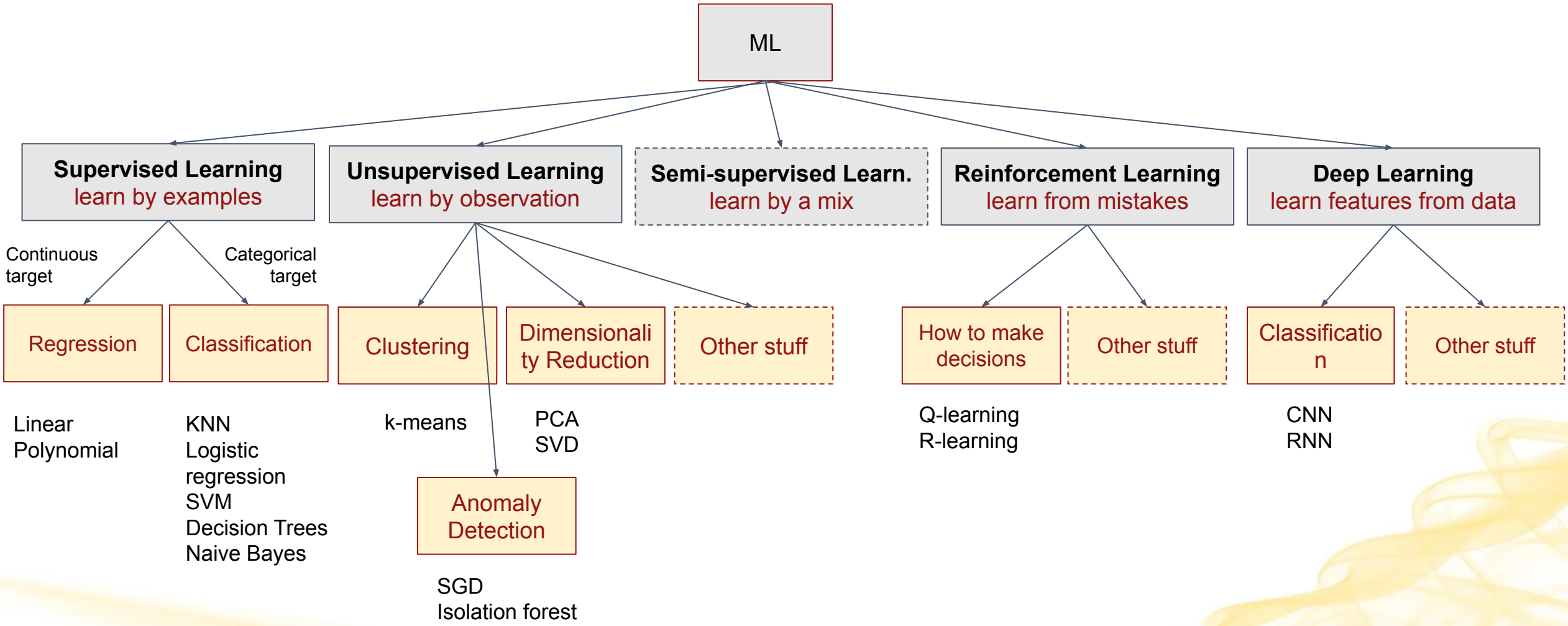
Unlabeled training dataset

Mix of Labeled & Unlabeled training dataset

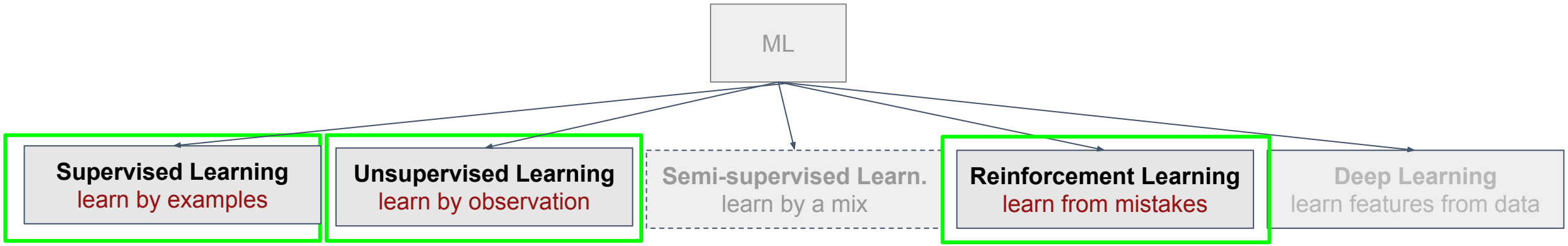
No training dataset. Uses trial and error and a reward function to learn.

Labeled or Unlabeled learning dataset - depends on context.

Main categories of ML Algorithms



Main categories of ML Algorithms



Let’s look at 3 fundamental categories.

Fundamental Categories

- **Supervised Learning**
  - The algorithm **learns from labeled data**
  - Each example in the training set is associated with a known target or output.
  - Used for tasks like classification and regression.
- **Unsupervised Learning**
  - The algorithm **learns from unlabeled data**
  - Tries to identify patterns or structures within the data, such as clustering similar data points or reducing data dimensionality.
- **Reinforcement Learning**
  - The algorithm **learns from mistakes/successes**.
  - Attempts to make a sequence of decisions or actions in an environment to maximize a reward signal (uses rewards and punishments to learn a task).
  - Often used in robotics, game playing, and autonomous systems.

Categories of Learning: Examples

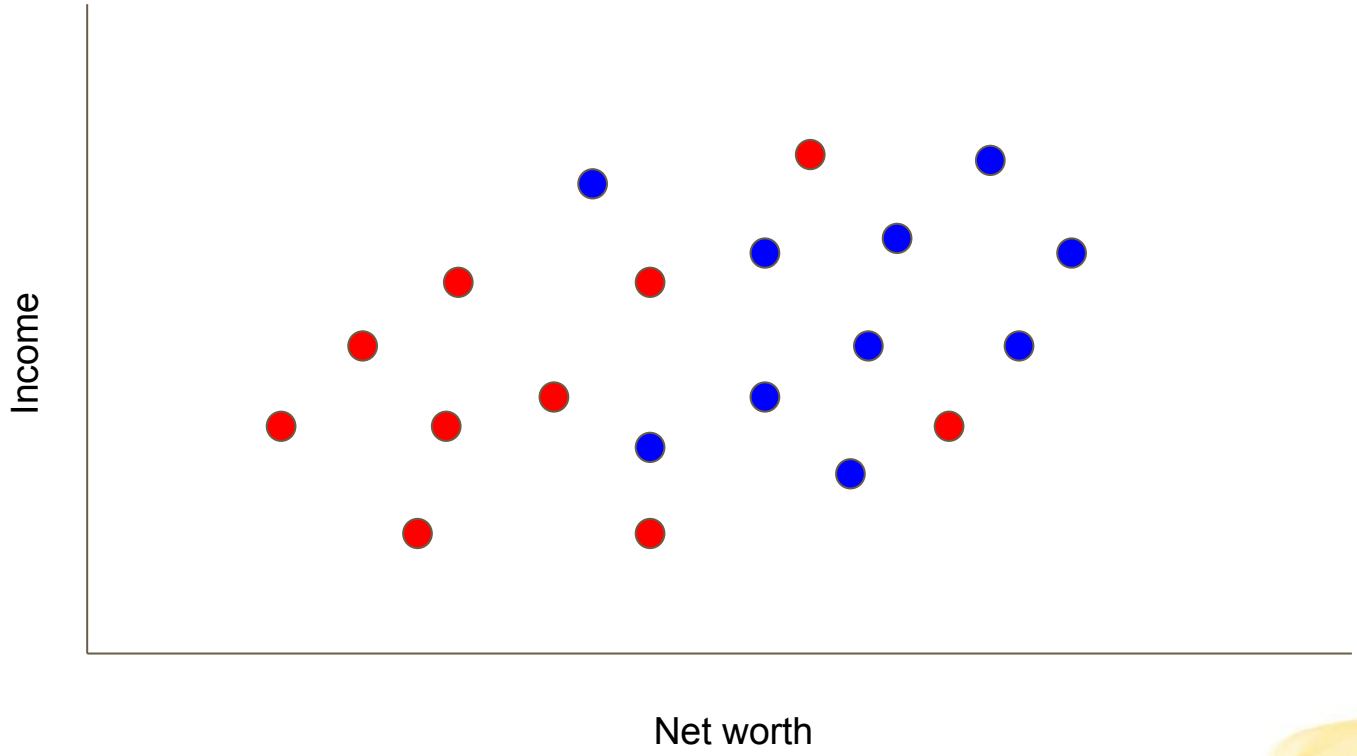
- **Supervised Learning**: learn to recognize whether an **email is spam or not** based on a dataset of labeled emails.
- **Unsupervised Learning**: Group **similar customer profiles** together in a retail dataset without knowing in advance what those groups represent.
- **Reinforcement Learning**: A computer learns by taking actions in an environment and receiving rewards. Think of a **self-driving car** learning to navigate roads safely by getting positive feedback (rewards) for following traffic rules and avoiding accidents.

Motivating Example

Motivating Example: Bank Loan Algorithm

Plot the datapoint:

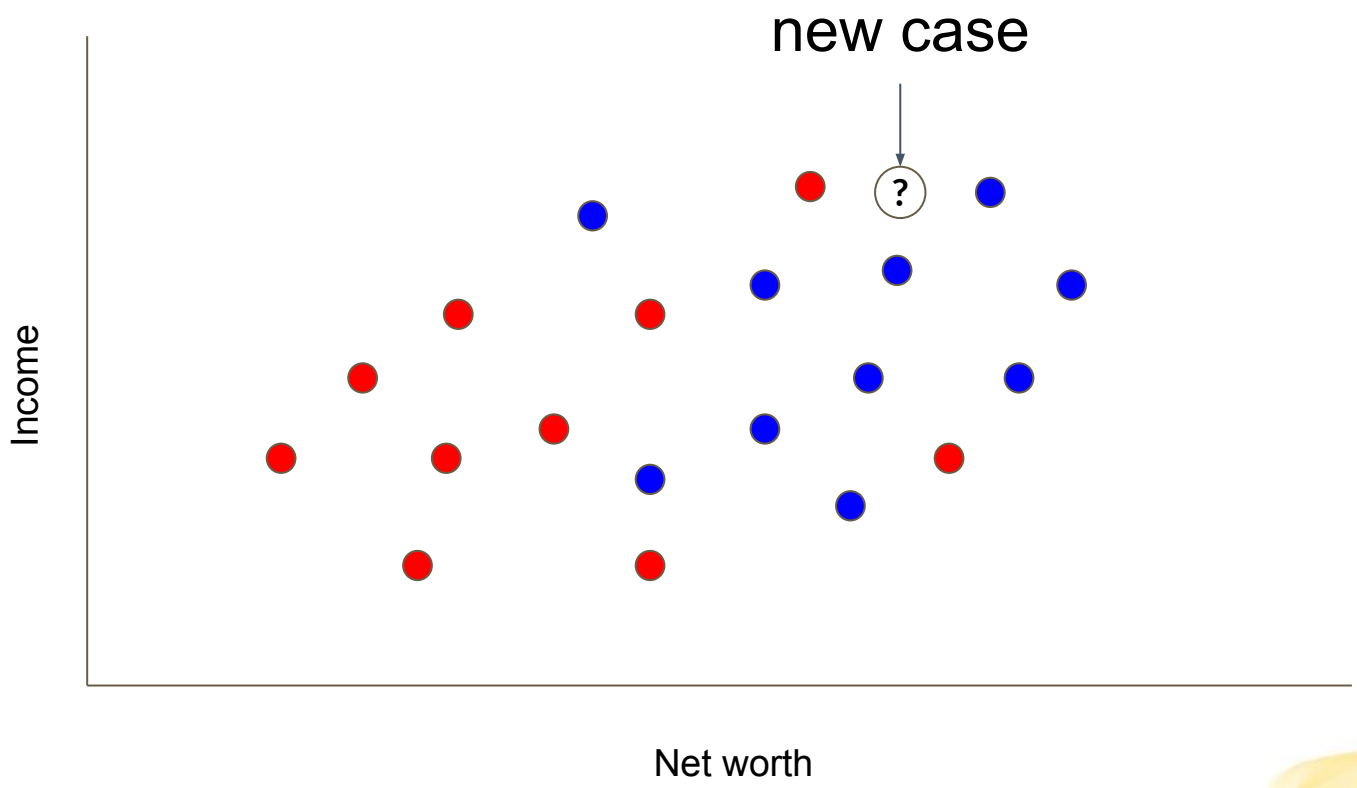
- Loan not paid
- Loan paid



Motivating Example: Bank Loan Algorithm

Plot the datapoint:

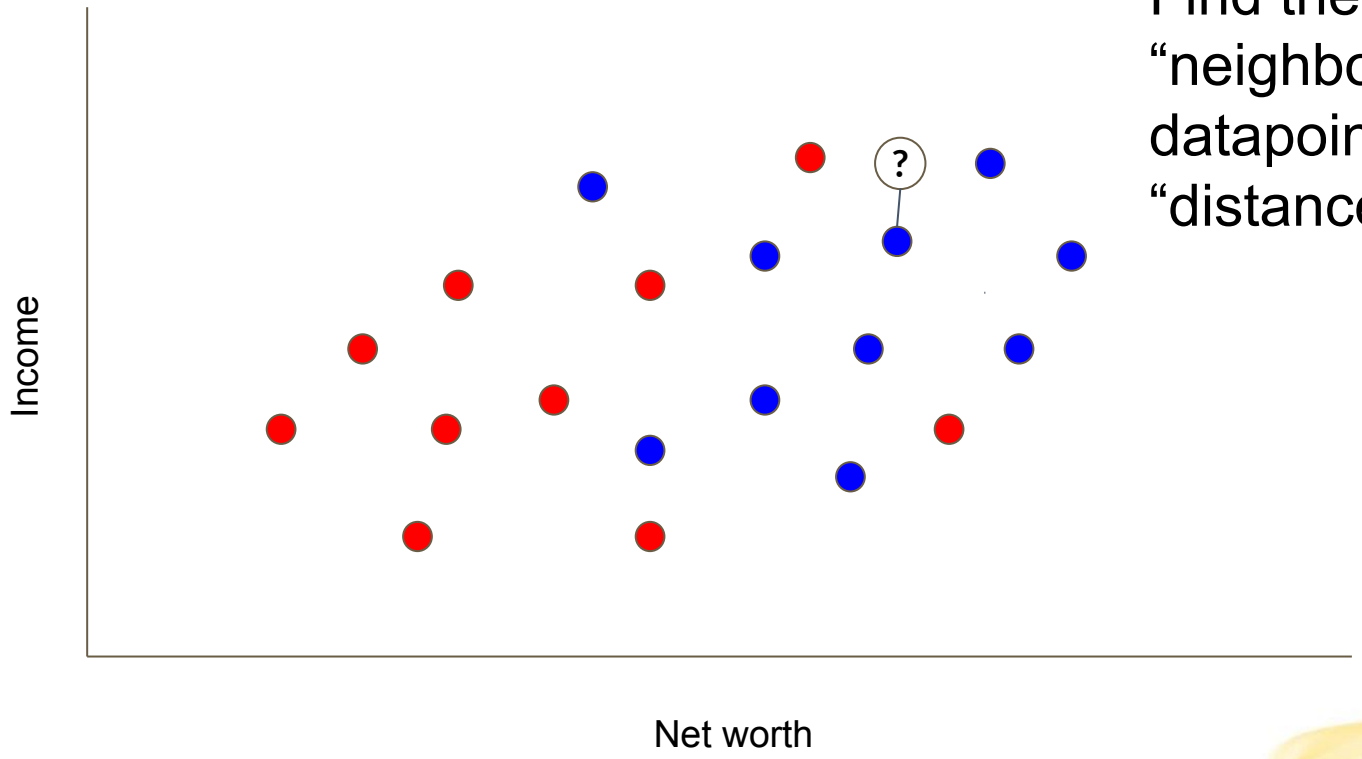
- Loan not paid
- Loan paid



Motivating Example: Bank Loan Algorithm

Plot the datapoint:

- Loan not paid
- Loan paid

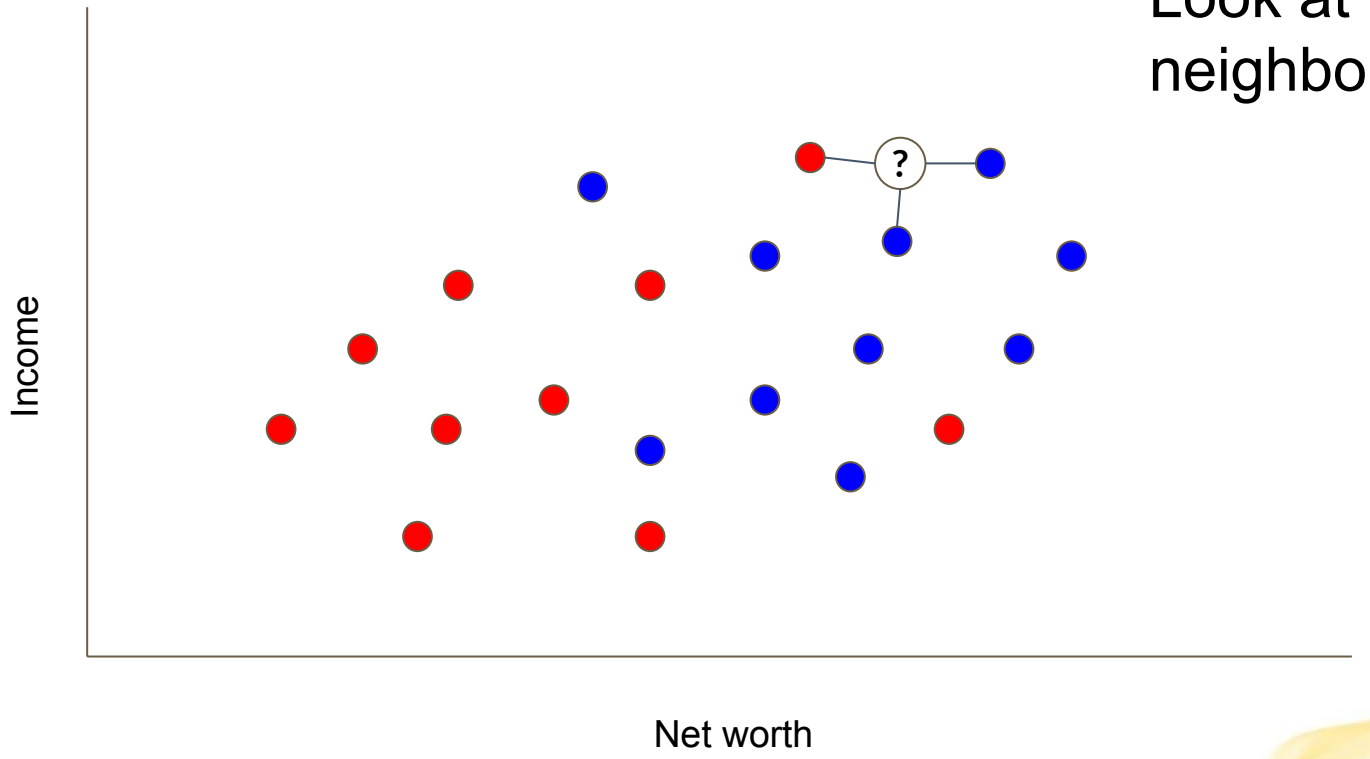


<u>1 method</u>:
Find the nearest "neighbor" for the new datapoint based on "distance"

Motivating Example: Bank Loan Algorithm

Plot the datapoint:

- Loan not paid
- Loan paid

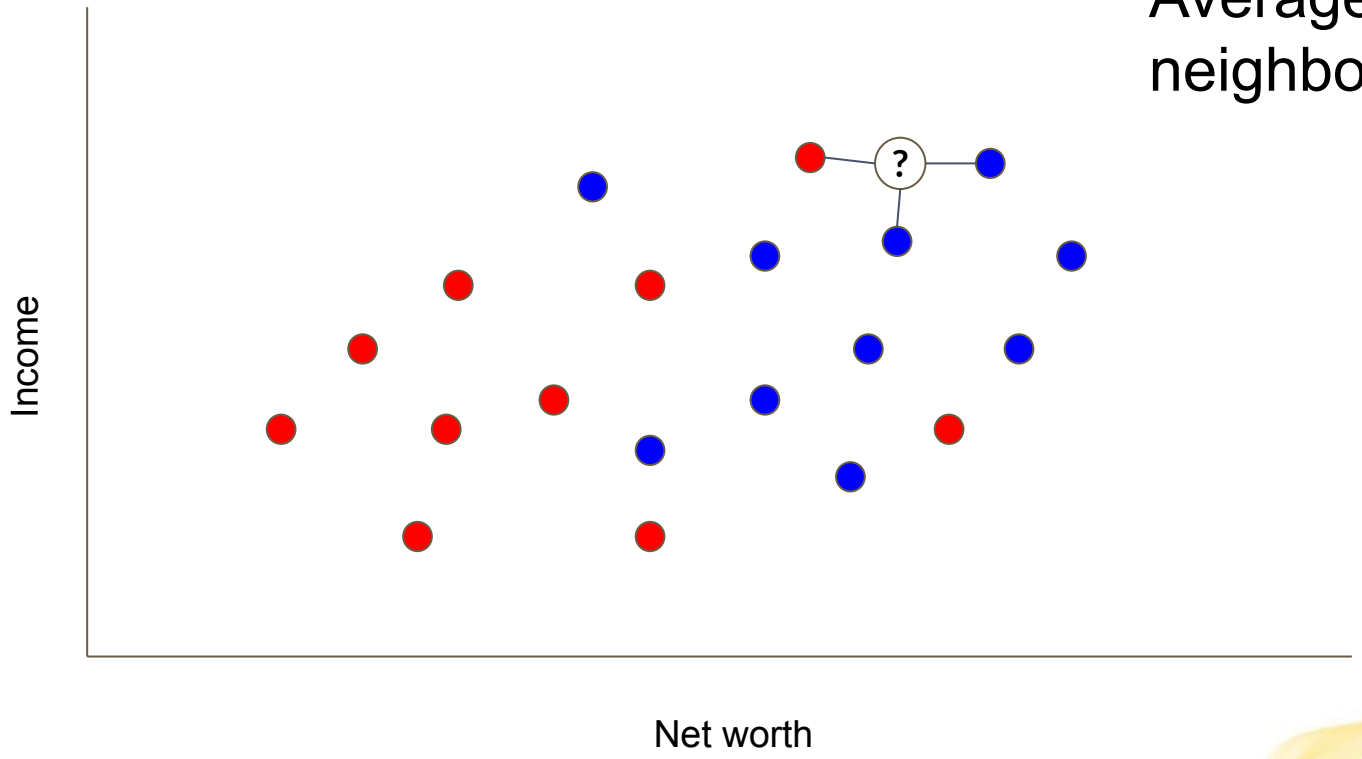


<u>Improve accuracy (?):</u>
Look at more than one neighbour

Motivating Example: Bank Loan Algorithm

Plot the datapoint:

- Loan not paid
- Loan paid

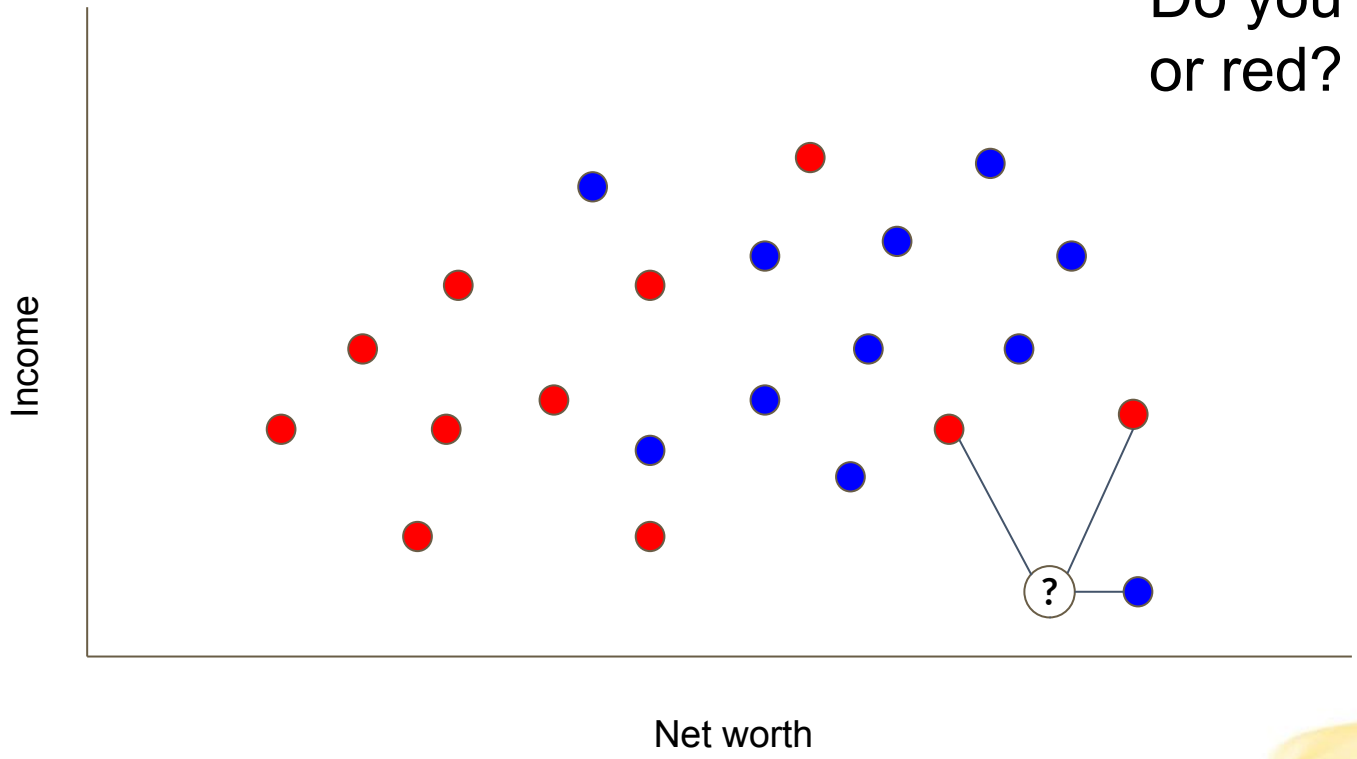


<u>Take an average:</u>
Average the 3 nearest neighbors.

Motivating Example: Bank Loan Algorithm

Plot the datapoint:

- Loan not paid
- Loan paid

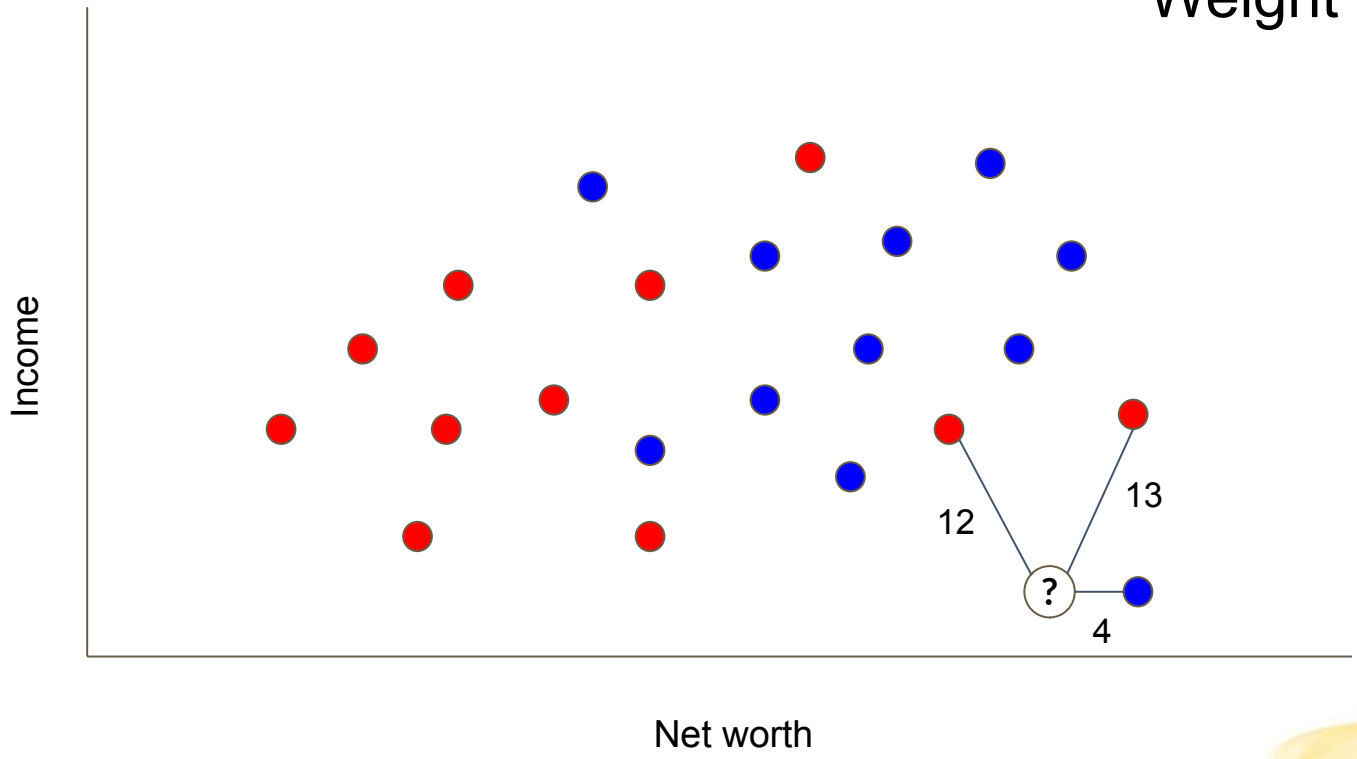


What about this?
Do you classify as blue or red?

Motivating Example: Bank Loan Algorithm

Plot the datapoint:

- Loan not paid
- Loan paid



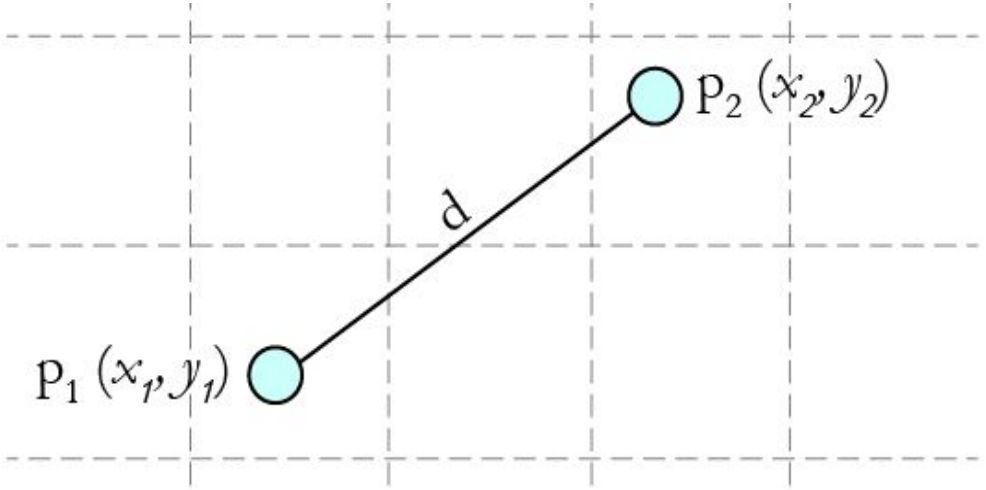
<u>One method:</u>
Weight by distance

Distance

What do we mean by distance?

Distance is a crucial concept in many algorithms!

- It measures the similarity or proximity between data points.
  - the numerical measure of how far apart two data points are in the feature space.



Why does Distance matter?

Many ML models depend on the distance between two data points to predict their output.

In the previous example, the distance points determined which points were the 3 nearest neighbors of the new point.

Why does Distance matter?

Many ML models depend on the distance between two data points to predict their output.

In the previous example, the distance between a data point and its k=3 nearest neighbors determined the weighting.

- Closer neighbors have higher influence, while distant neighbors have less.

Different kinds of distance functions

The choice of distance metric depends on the nature of the data and the problem at hand.

Some common distance functions:

- Euclidean Distance
- Manhattan Distance
- Minkowski Distance
- Hamming distance
- Cosine Similarity

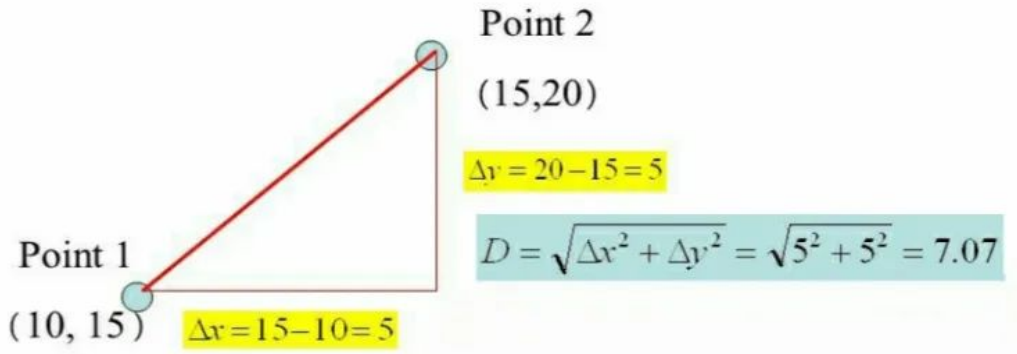
There are many others! Which one you choose depends on what you're trying to do.

Euclidean Distance (L2 distance)

Represents the shortest distance between two points.

- Most frequently used distance metric in machine learning.
- Measures the geometric (straight-line) distance between points, considering differences along each dimension.
- Best for continuous numeric data, as it considers the magnitude or size of differences.
  - Larger differences have a greater impact on the calculated distance.
- **Advantage**: Easy to understand
- **Disadvantage**: Sensitive to outliers

$$d(x,y)\left(\sum_{i=1}^{n}(x_i-y_i)^2\right)^{\frac{1}{2}}=\sqrt{\sum_{i=1}^{n}(x_i-y_i)^2}$$



Hamming Distance

Given two items, count the **number of positions with differences**.

- Used for strings/vectors (ex. comparing binary strings or categorical data)
- Commonly applied in text classification and DNA sequence analysis.

More examples (from Wikipedia)

- "**ka**r**o**l**in**" and "**ka**t**h**r**in**" is 3.
- "**ka**r**o**l**in**" and "**k**e**r**s**t**in**" is 3.
- "**ka**t**h**r**in**" and "**k**e**r**s**t**in**" is 4.
- **0000** and **1111** is 4.
- **2**17**3**8**96** and **2**23**3**7**96** is 3.

tern – turn or term?

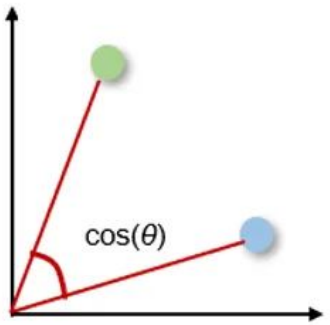
Cosine Similarity

A metric used to measure how similar two vectors are, especially in high-dimensional spaces.

Calculates the cosine of the angle between the two vectors (from the origin) in a multi-dimensional space.

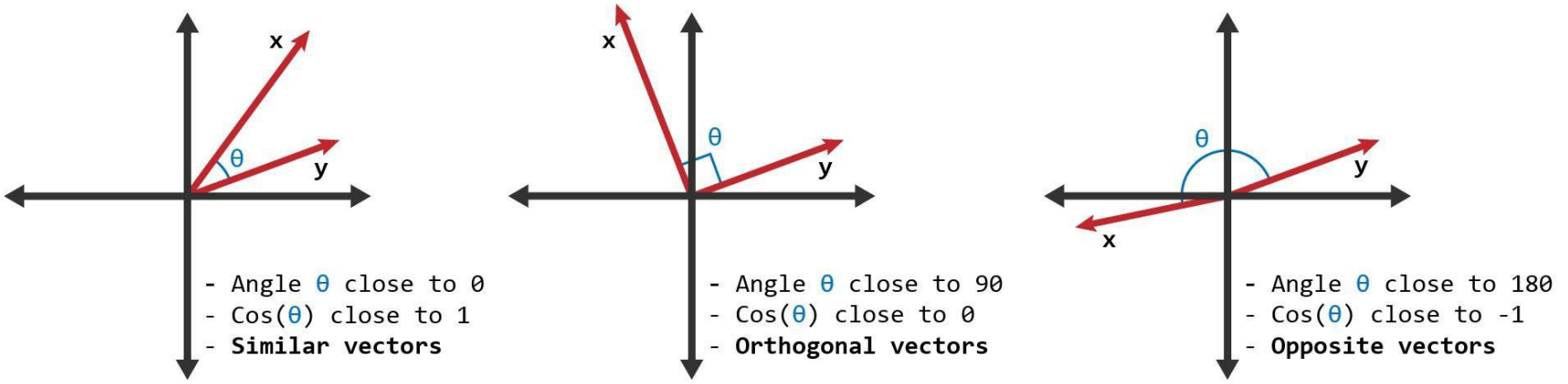
- A measure of orientation in the range from **-1 to 1**
- It's the dot product of the vectors, divided by the product of their lengths:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

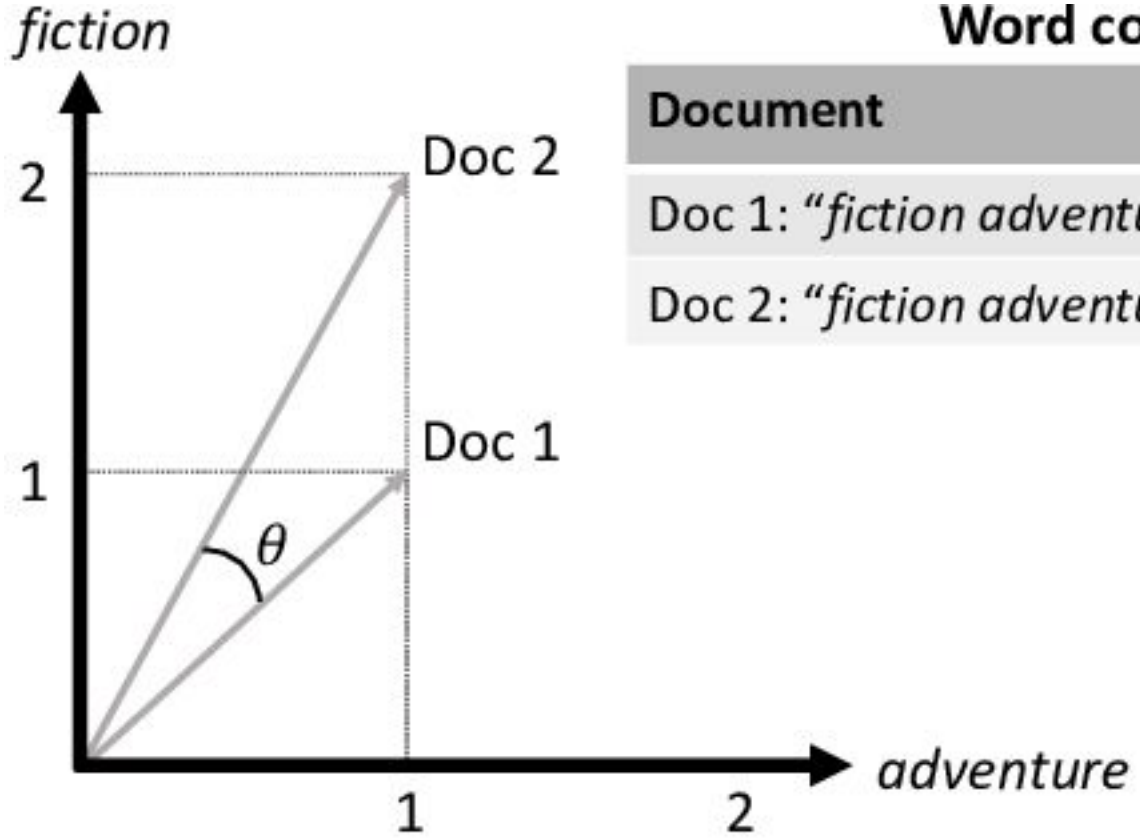


Cosine Similarity (cont.)

- **Purpose**: Calculate distance, measured as the cosine of the angle between the vectors
  - How similar of a direction are they each pointing in?
- **Applications**: Commonly used for comparing the similarity of documents (as vectors), text data, and high-dimensional data.
  - Generally works well for arbitrary vectors
- **Range**: Values range from -1 (completely dissimilar or pointing in opposite directions) to 1 (identical or similar in direction).



Cosine Similarity Example



Word count per document

Document	*fiction*	*adventure*
Doc 1: "*fiction adventure*"	1	1
Doc 2: "*fiction adventure fiction*"	2	1

$$\cos\theta = \frac{\overrightarrow{Doc\ 1} \cdot \overrightarrow{Doc\ 2}}{\left\|\overrightarrow{Doc\ 1}\right\| \left\|\overrightarrow{Doc\ 2}\right\|}$$

$$= \frac{(1 * 2) + (1 * 1)}{\sqrt{1^2 + 1^2}\sqrt{2^2 + 1^2}}$$

$$= 0.95$$

Summary: The choice of distance metric

The choice of distance metric depends on the problem, data type, and dimensionality.

- Euclidean Distance for continuous numerical data.
- Hamming Distance for things like string comparisons.
- Cosine Similarity for text mining, recommendation systems, and clustering.

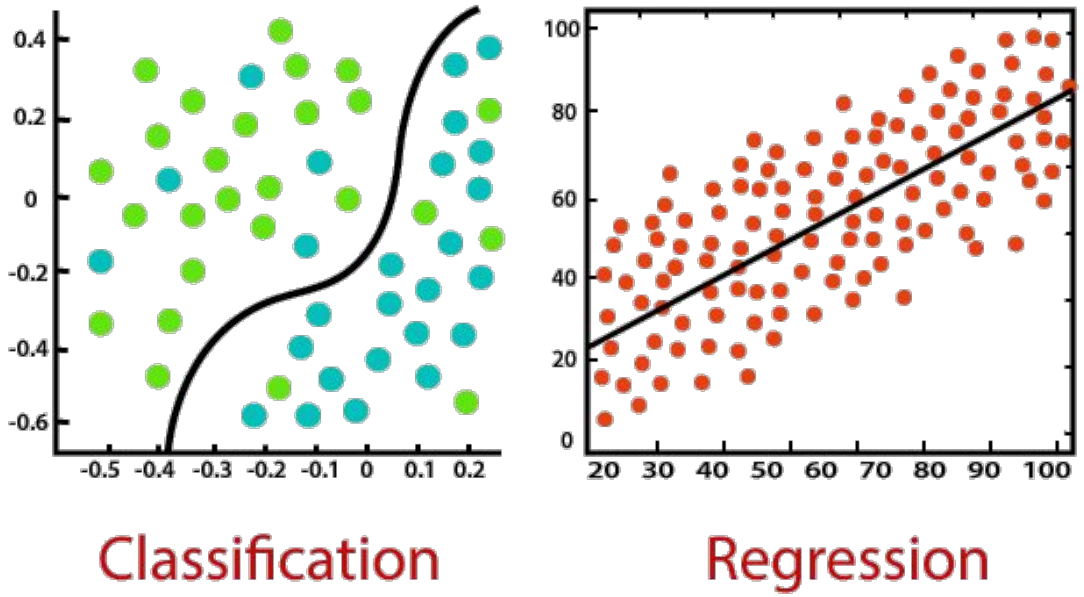
There are other kinds of distance too!

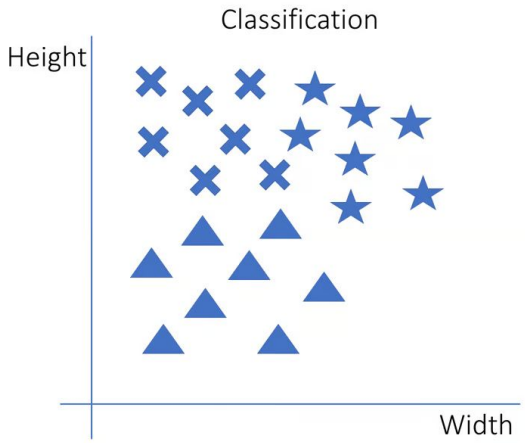
KNN

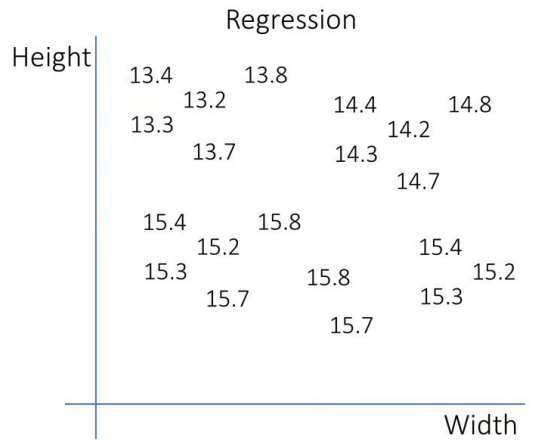
Our Example Bank Loan Algorithm: KNN

- Our previous Bank Loan example was actually a simple implementation of the K-Nearest Neighbors (KNN) **classification** algorithm
- KNN is simple, easy-to-implement **supervised** machine learning algorithm.
  - Predicts outcomes based on similarity or proximity between data points.
- Different variations of KNN can be used to solve both **classification** and **regression** problems.
  - Regression → Predict continuous numerical values
  - Classification → Predict a category

Classification vs Regression Visuals







Intuition for Nearest Neighbor Classification

Simple idea

- Store all training examples
- Classify new examples based on most similar training examples

Remember, KNN predicts outcomes based on the **similarity or proximity between data points**.

Classifier Definition

In ML, we call classification algorithms (predict a category, or class) "**classifiers**."

This term is a bit loose, as "classifier" can also at times refer to the model that outputs the category/class/label.

Components of a KNN Classifier

- **Distance metric**
  - How do we measure distance between data points/ instances?
- **The k hyperparameter**
  - How large a neighborhood should we consider?
  - **The "K" in KNN:** a user-defined parameter that specifies the number of neighboring data points to consider when making predictions.
    - **Choosing "K"**: K is typically an odd number to prevent ties in decision-making.

Hyperparameter Definition

- A **hyperparameter** is an argument to the model that determines how the model behaves
- It is a configurable parameter that controls a model's learning process.
- Hyperparameters are set before the learning process begins, and are different from *parameters*, which are learned automatically during the process.
  - For example, K
  - It's like a setting for the model.
  - It decides how the model behaves. For KNN, it tells us how many similar examples to consider when making a decision.

Hyperparameters vs Parameters

Hyperparameters

- Adjustable parameters that are set/tuned manually beforehand
- Used to help estimate model parameters but are independent of the training data
- Not part of the learned model

(Model) Parameters

- The “fitted parameters”
- Estimated based on training data (historical data) → the properties of the training data that are learned
- Part of the model

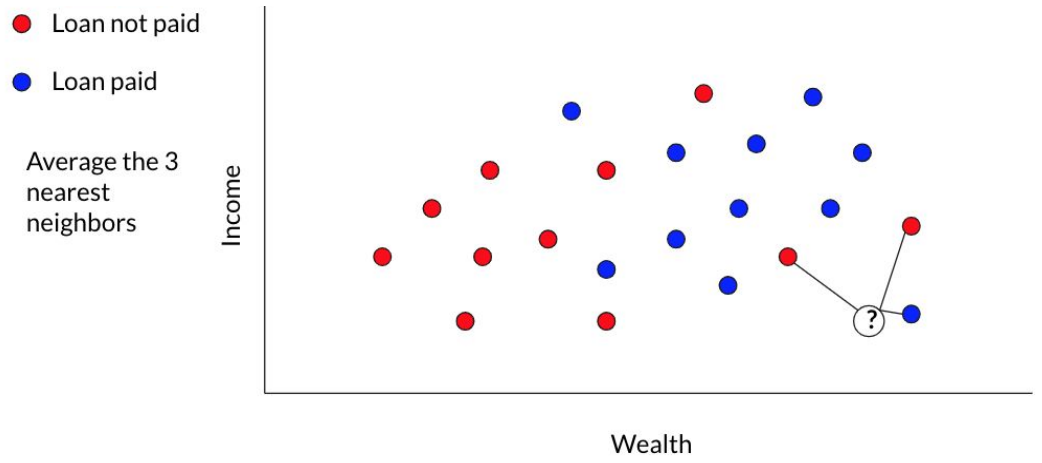
KNN Bank Loan Classification

Given the set K of the K nearest neighbors and point p:

- KNN Algorithm: identify the K most similar individuals (neighbors) to the person for whom we want to make a prediction (point p).

Let's make labels numeric:

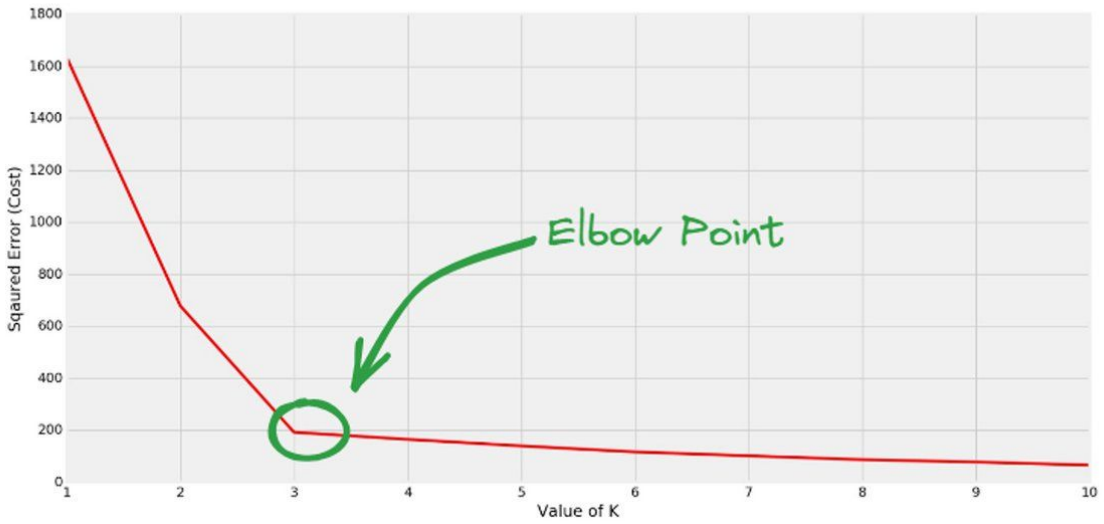
- Label -1: Represents someone who did not pay back the loan.
- Label 1: Represents someone who did pay back the loan.



How to Choose K in KNN

One way: Try multiple Ks and graph the variation, then pick the lowest inflection point, or elbow point.

- This is known as the **Elbow Method**
- The inflection point is not crisply defined, but this can be a useful way to identify an appropriate K value for your specific problem.



Variations on KNN: Weighted voting

Weighted Voting Default

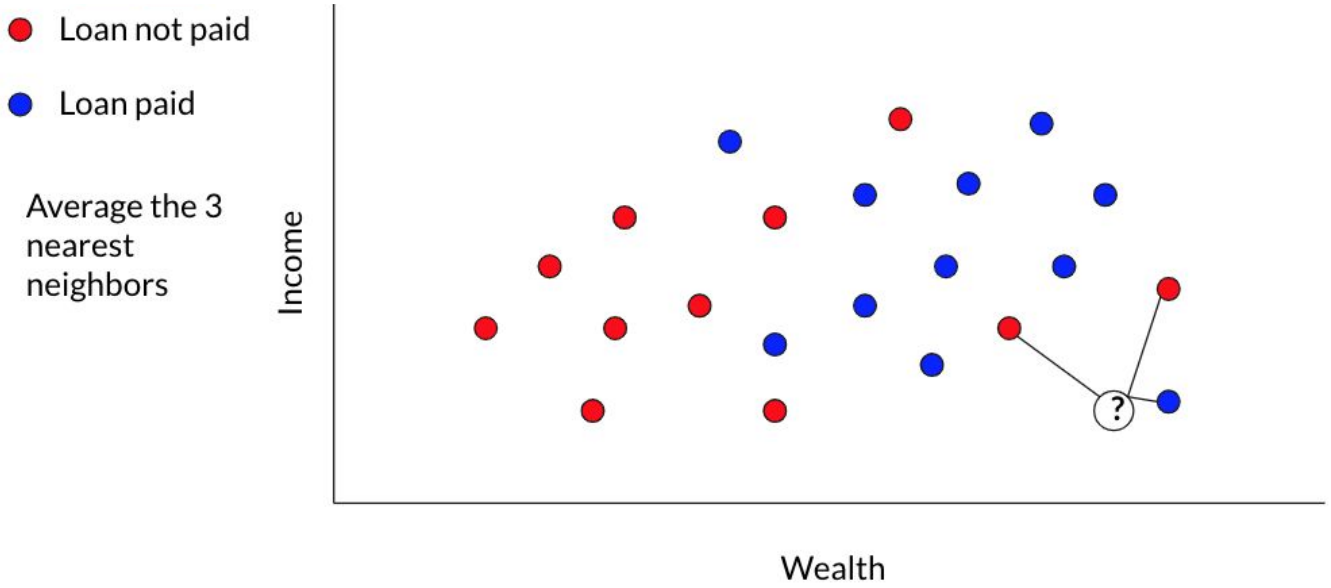
- All neighbors have equal weight
- Extension: weight neighbors by (inverse) distance
  - NEXT TOPIC: Weighted KNN

A Modified Version: Weighted KNN

In **Standard KNN**, prediction is based on the majority class among the K nearest data points and **equal weight** is given to **all K nearest neighbors**.

- Ex: If K=5 and the nearest neighbors are {1,1,1,−1,−1}, the prediction would be 1 (majority class).

Weighted KNN is similar to KNN, but the nearest k points are assigned a weight based on their distance.

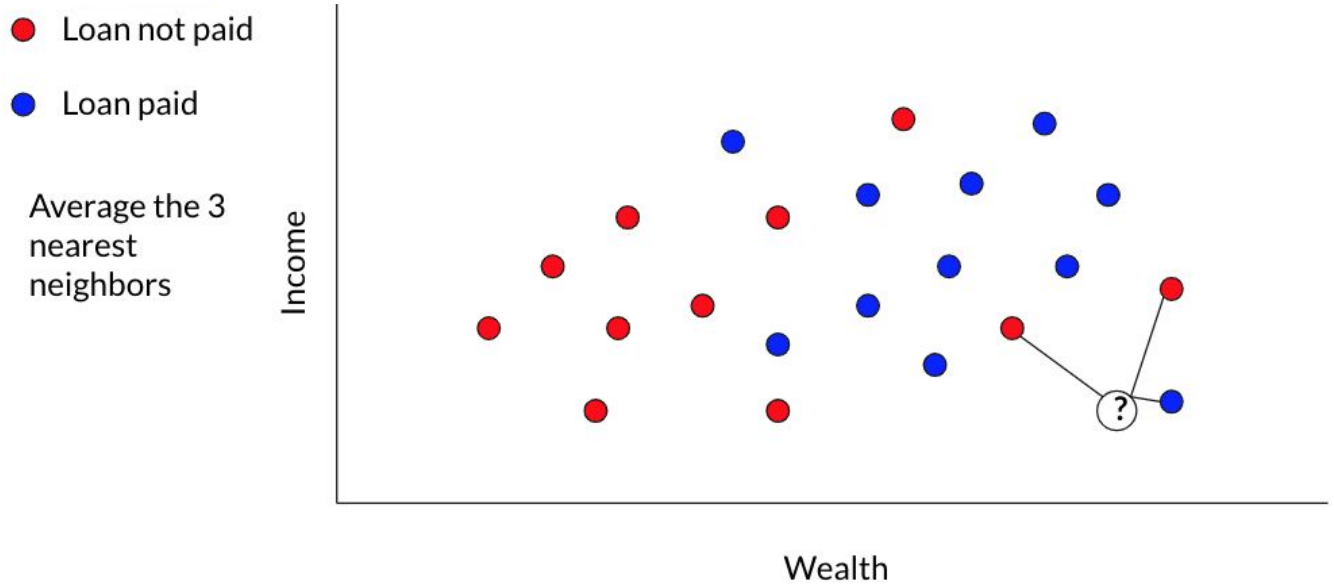


A Modified Version: Weighted KNN

In **Standard KNN**, prediction is based on the majority class among the K nearest data points and **equal weight** is given to **all K nearest neighbors**.

- Ex: If K=5 and the nearest neighbors are $\{1,1,1,-1,-1\}$, the prediction would be 1 (majority class).

The intuition behind weighted KNN is to give more weight to the points which are nearby and less weight to the points which are farther away.



Summary: KNN vs Weighted KNN

KNN	Weighted KNN
It treats **all neighbors equally**	Instead of treating all k-nearest neighbors equally, **assign different weights** to them based on their distance from the query point.
Classification tasks: count the occurrences of each class among the k-nearest neighbors and choose the class with the **highest count (mode)** as the predicted class.	**Classification tasks**: when determining the predicted class, the contributions of neighbors are scaled by their weights. *Closer neighbors have a stronger influence on the prediction.*
Regression tasks: take the **average (mean)** of the values associated with the k-nearest neighbors as the predicted value.	**Regression tasks**: the values associated with neighbors are multiplied by their weights before averaging, giving *more importance to closer neighbors in the prediction*.

Weighted KNN Equation

Weighted KNN not only considers the nearest neighbors, but also how close or far away they are, and assigns **different weights** to the K nearest neighbors **based on their distances to point p**.

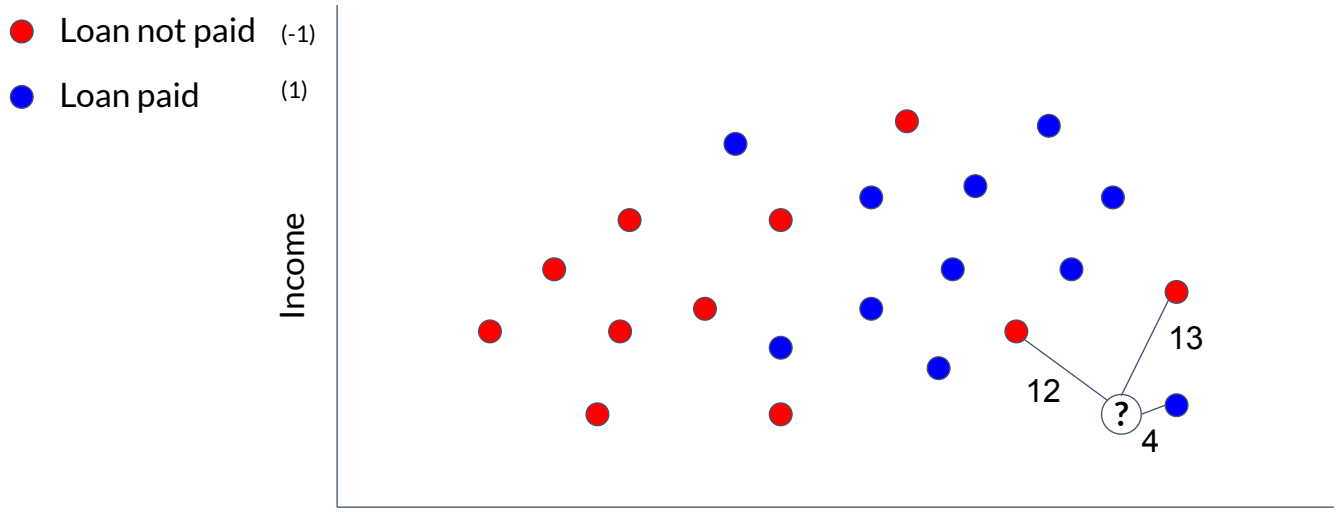
- How to assign? the weights are inversely proportional to the distances
- Closer neighbors have a stronger influence on the prediction.
  - Why? closer neighbors get higher weights, while farther neighbors get lower weights

$$y = \sum_{k \in K} \frac{1}{dist(k, p)} * k_{label}$$

predicted label or value for a new data point p.

label (or value) of the k-th nearest neighbor.

KNN Bank Loan Classification



Input

$$\frac{1}{12}\times(-1)+\frac{1}{13}\times(-1)+\frac{1}{4}\times 1$$

Exact result

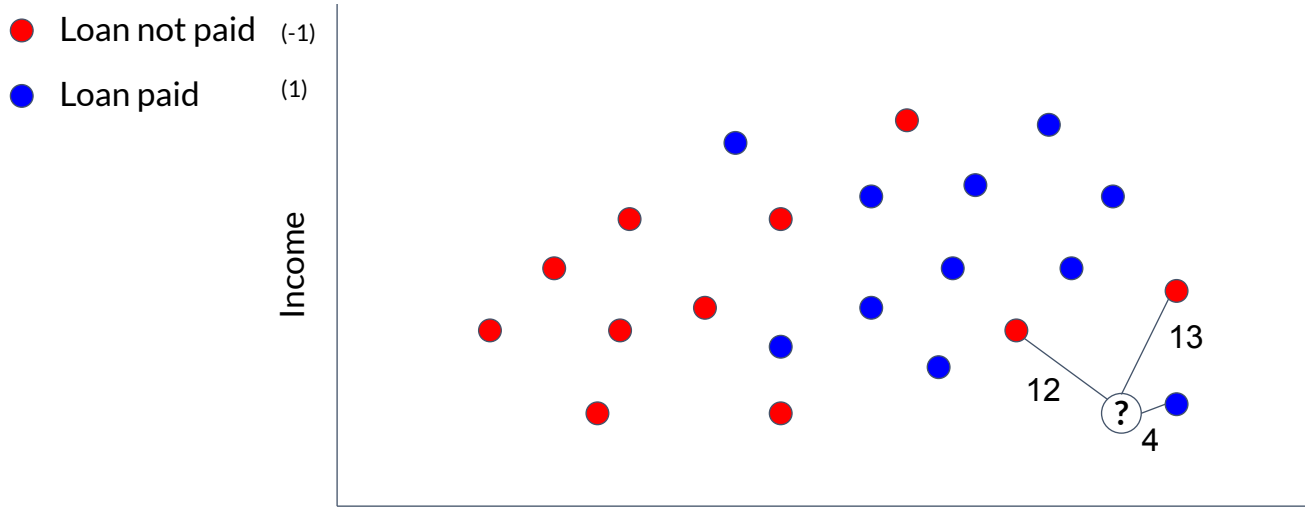
$$\frac{7}{78}$$

Decimal approximation

0.089743589743589743589743589

$$y = (\frac{1}{bigDistance} * -1) + (\frac{1}{bigDistance} * -1) + (\frac{1}{smallDistance} * 1)$$

KNN Bank Loan Classification



"Votes" for red:

$$\frac{1}{12} + \frac{1}{13} = \frac{25}{156} \approx 0.16$$

"Votes" for blue:

$$\frac{1}{4} = 0.25$$

Red wins.

Decision Boundary of a Classifier

The line that separates positive and negative regions in the feature space.

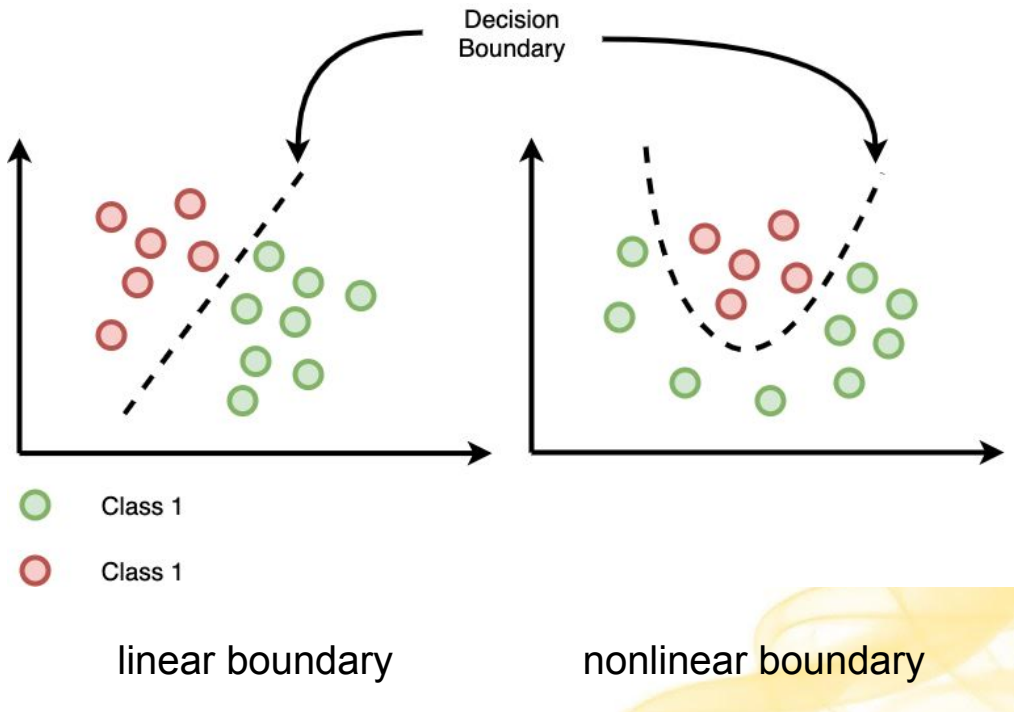
Purpose: It determines how new data points are classified based on their features.

Types:

- **Linear Boundary:** Straight line that separates classes in a linearly separable problem.
- **Non-linear Boundary:** Complex shapes that separate classes in non-linearly separable problems.

Why is it useful?

- Helps us visualize how examples will be classified for the entire feature space
- Helps us visualize the complexity of the learned model



KNN Boundary Conditions

What if K=1?

- Our KNN model considers only at 1 nearest neighbor (the closest) when making predictions.
  - **Decision Boundary:** The decision boundary can be irregular and jagged because the model is sensitive to individual data points.
- Our model will basically be almost memorizing things - it will just say whatever is closest.
  - **Doesn't consider any broader patterns or relationships in the data.**

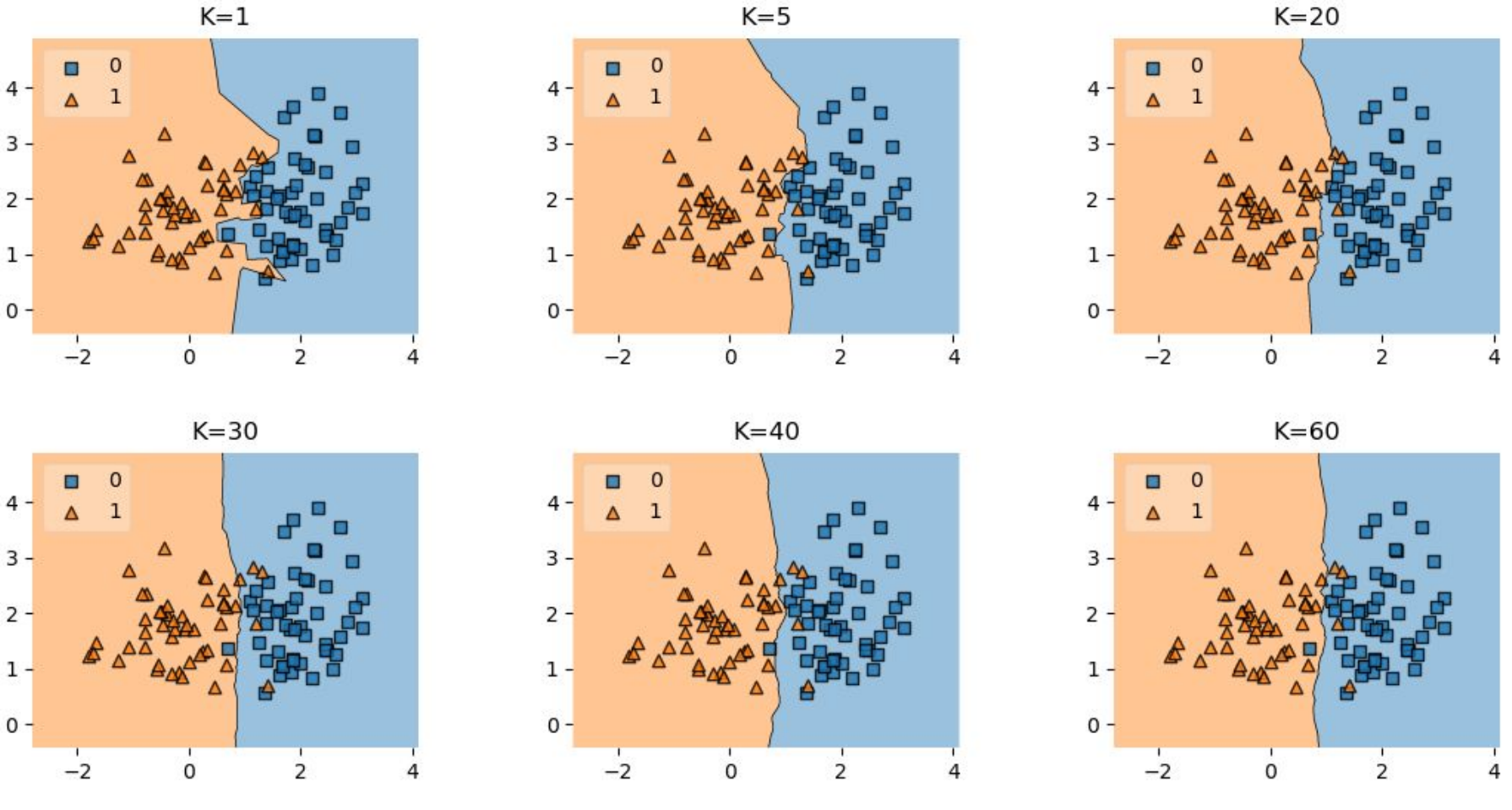
What if K=N?

KNN Boundary Conditions

What if K=N?

- Our KNN model **considers all data points as neighbors**, effectively taking into account the entire dataset.
  - It treats all data points as relevant neighbors, regardless of their actual similarity to the query point.
  - It ignores the local structure of the data.
- Soon, more and more irrelevant points will contribute, and we will just predict the mode of the data for any new data point.

KNN Boundary Conditions Visualized



Pros and Cons of K-Nearest Neighbor

Pros

- **Pretty accurate**: For many applications, KNN does a great job. Used successfully in large-scale production in industry for a variety of real-life problems.
- **Easy to implement**: Given the algorithm's simplicity and accuracy, it is one of the easiest to implement and one of the first classifiers a new data scientist will learn.
- **Easy to understand**: Very simple rule set that is highly explainable. All you need to understand is distance and how you got k.
- **Few hyperparameters**: KNN only requires a k value and a distance metric, which is low compared to other machine learning algorithms.
- **Lazy learning**: KNN is a **lazy algorithm**, meaning that it doesn't have a training step because it stores the entire dataset in memory and uses it directly to make predictions.

Pros and Cons of K-Nearest Neighbor

Cons

- **Resources**: Requires higher memory and has higher computational cost compared to other classifiers.
  - Needs to store and process the entire training set, which can be very large (on the order of petabytes or exabytes).
- **Tuning**: K must be chosen/tuned correctly and a proper distance metric chosen.
  - Choice of K impacts performance of the KNN model and needs to be selected carefully
    - K too small → noise and outliers effect
    - K too large → lose relevant info for effective choice of boundary
  - Choice of distance metric must be appropriate for the specific problem

Summary: Vocabulary for KNN

- KNN is a **supervised learning** model, which means the **training data is labeled**.
  - For our example, we had labels for whether or not a person had paid back their loans.
- Our bank loan example was a **classification** problem, where we were deciding between categories
  - We sorted things into categories: we were deciding if someone belongs to one of two groups (Pay Back or Not).
- We did **binary classification** where there are only two categories.
- KNN is an **instance-based model**, meaning it requires all the training data be stored in memory to work.
- We chose appropriate **hyperparameters**, which are the parameters that need to be **tuned** ahead of time in order to run the algorithm
  - For KNN, is simply the value of K.
  - We also chose a distance metric, though that didn't need to be tuned

Python Code Sample for KNN

1. Import necessary modules

```
from sklearn.neighbors import KNeighborsClassifier
```

2. Load your labeled training data
3. Create a KNN model (for a predetermined value of K; default distance is Euclidean) and fit it to the training data:

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(data, labels)
```

4. Make a prediction on new data points that the model has not seen before

```
prediction = knn.predict(new_point)
```

Other algorithms

There are many, many ML algorithms.

And many, many varieties of each algorithm.

Here are *some* of them:

Averaged one-dependence estimators
Artificial neural network
Convolutional neural network
Extreme learning machine
Feedforward neural network
Logic learning machine
Long short-term memory
Recurrent neural network
Self-organizing map
Bayesian networks
Boosting
Case-based reasoning
Conditional random field
C4.5 algorithm
C5.0 algorithm
Chi-squared automatic interaction detection
Classification and regression tree
Conditional decision tree
Decision stump
Decision tree
ID3 algorithm
Iterative dichotomiser 3
Random forest
SLIQ
Bootstrap aggregating
Boosting
Gaussian process regression
Gene expression programming
Group method of data handling
Inductive logic programming
Information fuzzy networks
Instance-based learning
Eclat algorithm

K-nearest neighbour
Lazy learning
Learning vector quantization
Elastic-net
Lasso
Linear discriminant analysis
Linear regression
Logistic regression
Multinomial logistic regression
Naive bayes classifier
Ordinary least squares
Passive aggressive algorithms
Perceptron
Polynomial regression
Ridge regression / classification
Support vector machine
Logistic model tree
Analogical modelling
Nearest neighbour algorithm
Ordinal classification
Probably approximately correct learning
Quadratic classifiers
Random forests
Ripple down rules
Symbolic machine learning
Active learning
Co-training
Graph-based methods
Generative models
Low-density separation
Transduction
Apriori algorithm

FP-growth algorithm
Auto-encoders
BIRCH
Conceptual clustering
DBSCAN
Expectation-maximization
Fuzzy clustering
Hierarchical clustering
K-means clustering
K-medians
Mean-shift
OPTICS algorithm
Single-linkage clustering
Canonical correlation analysis
Dynamic mode decomposition
Factor analysis
Feature extraction
Feature selection
Independent component analysis
Linear discriminant analysis
Multidimensional scaling
Non-negative matrix factorization
Partial least squares regression
Principal component analysis
Principal component regression
Projection pursuit
Sammon mapping
T-distributed stochastic neighbour embedding
Expectation-maximization algorithm
Generative topographic map
Information bottleneck method
Manifold learning

Deterministic policy gradient
Learning automata
Proximal policy optimization
Q-learning
Soft actor-critic
State–action–reward–state–action
Temporal difference learning
Trust region policy Optimization
Other

Bayesian belief network
Bayesian knowledge base
Deep belief networks
Deep boltzmann machines
Deep neural networks
Discrepancy modelling
Gaussian naive bayes
Generative adversarial network
Hierarchical temporal memory
Knowledge-enhanced machine learning
Markov models
Multinomial naive bayes
Neural style transfer
Physics-informed machine learning
Sparse identification of nonlinear dynamics
Transformer
Vector quantization

In this class

We'll look at a handful of the most common/important algorithms.

Understanding these will give you a good foundation and make it easier to learn others as needed.

No Free Lunch Theorem (NFL)

There is no single algorithm that will work well for all tasks.